

CCDS-221 Data Warehousing - Project Report

Member Group:

- Abdulaziz Yasiir Bugshan 2342952

Live SQL Session Links:

- Preliminary Work: https://freesql.com/library/scripts/project-preliminary-work-CH1KSv?share_key=McDvNJRniB
- Task 1: https://freesql.com/library/scripts/project-task-1-zEVxUR?share_key=jNSzoNegaG
- Task 2: https://freesql.com/library/scripts/project-task-2-z3BJLp?share_key=JhjmKlfu8u
- Task 3: https://freesql.com/library/scripts/project-task-3-Rv56GV?share_key=hGn8DHfoty

Table of Contents

Preliminary Work	4
I. Create ORDERS_COPY Table	4
1. Full Table Copy and Display	4
2. ORDER_MODE Capitalization (UPPER function)	10
3. Identification of NULL SALES_REP_ID entries.....	10
4. Row Selection for NULL SALES_REP_ID.....	11
5. Replacement of NULL IDs with 999 using NVL	13
6. Content Verification	13
7. PROMOTION_ID Count Analysis	18
8. Theoretical Analysis of COUNT behavior	18
9. Replacement of NULL PROMOTION_ID with 0 using NVL	19
10. Count Verification and Value Explanation.....	20
II. Create T_DATE Table.....	21
1. T_ORDER_DATE Staging Table Creation.....	21
2. ROWID and Content Display	22
3. Distinct Date Count and Frequency Analysis.....	25
4. Deduplication Methodology	27
5. T_DATE Table Structure Definition	29
6. Population via Extraction Functions (TO_CHAR/TO_NUMBER).....	30
III. Create CUSTOMER_COPY table	32
ETL Process - Task 1: Calculate the Turnover and the Type of the customer	33
1. Make a copy of the OE.CUSTOMER table	33
2. Addition of TURNOVER (12,2) Column	34
3. Addition of CUST_TYPE (VARCHAR) Column	35
4. Correlated Update Statement for Turnover Calculation	36
5. PL/SQL Block Implementation for Classification (Types A, B, and C)	38
ETL Process - Task 2: Currency Conversion C ORDERS_IN_EURO.....	41
1. Identification of Distinct Customer Territories.....	41
2. Development of ORDERS_IN_EURO Staging Table.....	44
3. PL/SQL Logic for Multi-Currency Euro Conversion.....	45
Task 3: Star Schema Implementation	47
1. Create T_SALESMAN table	48

2. Create Dimensional Tables	49
a) Handle D_ORDERS Table.....	49
b) Handle D_CUSTOMERS table	51
c) Handle D_TIME table.....	53
d) Handle D_SALESMAN table	56
3. Create Fact Table	58
a) Applying PK.....	59
b) Foreign Keys	60
c) Sequence for Fact Table.....	61
d) Load F_ORDERS	62
e) Display Sum of the ORDER_AMOUNT	63
f) Turnover by Customer and Year.....	64
g) Count orders > value	66
h) Display the Percentage of amount of DIRECT Orders and ONLINE Orders.....	67

Preliminary Work

I. Create ORDERS_COPY Table

1. Full Table Copy and Display

Use the CREATE and SELECT SQL statements to Create a Temporary table as a full copy (all rows & columns) of table ORDERS. Name the new table ORDERS_COPY.

```
CREATE TABLE ORDERS_COPY AS  
SELECT * FROM OE.ORDERS;
```

The screenshot shows an SQL Worksheet interface. The top section contains a toolbar with icons for running queries, saving, and other functions. Below the toolbar is a text area with the following SQL script:

```
1 /* 1. Use the CREATE and SELECT SQL statements to Create a Temporary table as a full copy (all  
2 rows & columns) of table ORDERS. Name the new table ORDERS_COPY. */  
3  
4 CREATE TABLE ORDERS_COPY AS  
5 SELECT * FROM OE.ORDERS;  
6  
7 SELECT * FROM ORDERS_COPY;  
8
```

Below the script area is a tabbed interface with the following tabs: Query result, Script output, DBMS output, Explain Plan, and SQL history. The 'Script output' tab is currently selected. It shows the following output:

```
SQL> CREATE TABLE ORDERS_COPY AS  
SELECT * FROM OE.ORDERS  
  
Table ORDERS_COPY created.  
Elapsed: 00:00:00.045
```

Table ORDERS_COPY created.

Display the content of the new table.

SELECT * FROM ORDERS_COPY;

[SQL Worksheet]*

▶
⌵
🔗
📄
⌵
Aa
🗑️

1 /* 1. Use the CREATE and SELECT SQL statements to Create a Temporary table as a full copy (all
2 rows & columns) of table ORDERS. Name the new table ORDERS_COPY. */
3
4 CREATE TABLE ORDERS_COPY AS
5 SELECT * FROM OE.ORDERS;
6
7 SELECT * FROM ORDERS_COPY;
8

Query result
Script output
DBMS output
Explain Plan
SQL history

🗑️ ⓘ Download ▾ Execution time: 0.141 seconds

	ORDER_ID	ORDER_DATE	ORDER_MODE	CUSTOMER_ID	ORDER_STATUS	ORDER_TOTAL	SALES_REP_ID	PROMC
1	2458	2007-08-16T14:34:12.234359Z	direct	101	0	78279.6	153	
2	2397	2007-11-19T14:41:54.696211Z	direct	102	1	42283.2	154	
3	2454	2007-10-02T16:49:34.67834Z	direct	103	1	6653.4	154	
4	2354	2008-07-14T17:18:23.234567Z	direct	104	0	46257	155	
5	2358	2008-01-08T17:03:12.654278Z	direct	105	2	7826	155	
6	2381	2008-05-14T19:59:08.843679Z	direct	106	3	23034.6	156	

ORDER_ID	ORDER_DATE	ORDER_M ODE	CUSTOME R_ID	ORDER_STA TUS	ORDER_TO TAL	SALES_REP _ID	PROMOTIO N_ID
2458	2007-08-16T14:34:12.234359Z	direct	101	0	78279.6	153	(null)
2397	2007-11-19T14:41:54.696211Z	direct	102	1	42283.2	154	(null)
2454	2007-10-02T16:49:34.67834Z	direct	103	1	6653.4	154	(null)
2354	2008-07-14T17:18:23.234567Z	direct	104	0	46257	155	(null)
2358	2008-01-08T17:03:12.654278Z	direct	105	2	7826	155	(null)
2381	2008-05-14T19:59:08.843679Z	direct	106	3	23034.6	156	(null)
2440	2007-08-31T20:53:06.008765Z	direct	107	3	70576.9	156	(null)
2357	2006-01-08T20:19:44.123456Z	direct	108	5	59872.4	158	(null)
2394	2008-02-10T21:22:35.564789Z	direct	109	5	21863	158	(null)
2435	2007-09-02T22:22:53.134567Z	direct	144	6	62303	159	(null)
2455	2007-09-20T10:34:11.456789Z	direct	145	7	14087.5	160	(null)
2379	2007-05-16T01:22:24.234567Z	direct	146	8	17848.2	161	(null)

2396	2006-02-02T01:34:56.345678Z	direct	147	8	34930	161	(null)
2406	2007-06-29T03:41:20.098765Z	direct	148	8	2854.2	161	(null)
2434	2007-09-13T04:49:30.647893Z	direct	149	8	268651.8	161	(null)
2436	2007-09-02T05:18:04.378034Z	direct	116	8	6394.8	161	(null)
2446	2007-07-27T06:03:08.302945Z	direct	117	8	103679.3	161	(null)
2447	2008-07-27T07:59:10.223344Z	direct	101	8	33893.6	161	(null)
2432	2007-09-14T08:53:40.223345Z	direct	102	10	10523	163	(null)
2433	2007-09-13T09:19:00.654279Z	direct	103	10	78	163	(null)
2355	2006-01-26T09:22:51.962632Z	online	104	8	94513.5	(null)	(null)
2356	2008-01-26T09:22:41.934562Z	online	105	5	29473.8	(null)	(null)
2359	2006-01-08T21:34:13.112233Z	online	106	9	5543.1	(null)	(null)
2360	2007-11-14T12:22:31.223344Z	online	107	4	990.4	(null)	(null)
2361	2007-11-13T13:34:21.98621Z	online	108	8	120131.3	(null)	(null)
2362	2007-11-13T14:41:10.619477Z	online	109	4	92829.4	(null)	(null)
2363	2007-10-23T16:49:56.346122Z	online	144	0	10082.3	(null)	(null)
2364	2007-08-28T17:18:45.942399Z	online	145	4	9500	(null)	(null)
2365	2007-08-28T18:03:34.003399Z	online	146	9	27455.3	(null)	(null)
2366	2007-08-28T19:59:23.144778Z	online	147	5	37319.4	(null)	(null)
2367	2008-06-27T20:53:32.335522Z	online	148	10	144054.8	(null)	(null)
2368	2008-06-26T21:19:43.190089Z	online	149	10	60065	(null)	(null)
2369	2007-06-26T22:22:54.009932Z	online	116	0	11097.4	(null)	(null)
2370	2008-06-26T23:22:11.647398Z	online	117	4	126	(null)	(null)
2371	2007-05-16T00:34:56.113356Z	online	118	6	79405.6	(null)	(null)
2372	2007-02-27T00:22:33.356789Z	online	119	9	16447.2	(null)	(null)
2373	2008-02-27T01:34:51.220065Z	online	120	4	416	(null)	(null)
2374	2008-02-27T02:41:45.109654Z	online	121	0	4797	(null)	(null)

2375	2007-02-26T03:49:50.459233Z	online	122	2	103834.4	(null)	(null)
2376	2007-06-07T05:18:08.88331Z	online	123	6	11006.2	(null)	(null)
2377	2007-06-07T06:03:01.0011Z	online	141	5	38017.8	(null)	(null)
2378	2007-05-24T07:59:10.010101Z	online	142	5	25691.3	(null)	(null)
2380	2007-05-16T08:53:02.90909Z	online	143	3	27132.6	(null)	(null)
2382	2008-05-14T09:19:03.828321Z	online	144	8	71173	(null)	(null)
2383	2008-05-12T10:22:30.545103Z	online	145	8	36374.7	(null)	(null)
2384	2008-05-12T11:22:34.525972Z	online	146	3	29249.1	(null)	(null)
2385	2007-12-08T11:34:11.331392Z	online	147	4	295892	(null)	(null)
2386	2007-12-06T12:22:34.225609Z	online	148	10	21116.9	(null)	(null)
2387	2007-03-11T14:34:56.536966Z	online	149	5	52758.9	(null)	(null)
2388	2007-06-04T15:41:12.554435Z	online	150	4	282694.3	(null)	(null)
2389	2008-06-04T16:49:43.546954Z	online	151	4	17620	(null)	(null)
2390	2007-11-18T16:18:50.546851Z	online	152	9	7616.8	(null)	(null)
2391	2006-02-27T17:03:03.82833Z	direct	153	2	48070.6	156	(null)
2392	2007-07-21T19:59:57.571057Z	direct	154	9	26632	161	(null)
2393	2008-02-10T19:53:19.528202Z	direct	155	4	23431.9	161	(null)
2395	2006-02-02T20:19:11.22755Z	direct	156	3	68501	163	(null)
2398	2007-11-19T21:22:53.224175Z	direct	157	9	7110.3	163	(null)
2399	2007-11-19T22:22:38.34099Z	direct	158	0	25270.3	161	(null)
2400	2007-07-10T00:34:29.559387Z	direct	159	2	69286.4	161	(null)
2401	2007-07-10T01:22:53.554822Z	direct	160	3	969.2	163	(null)
2402	2007-07-02T02:34:44.66517Z	direct	161	8	600	154	(null)
2403	2007-07-01T15:49:13.615512Z	direct	162	0	220	154	(null)
2404	2007-07-01T15:49:13.664085Z	direct	163	6	510	158	(null)
2405	2007-07-01T15:49:13.678123Z	direct	164	5	1233	159	(null)

2407	2007-06-29T06:03:21.526005Z	direct	165	9	2519	155	(null)
2408	2007-06-29T07:59:31.333617Z	direct	166	1	309	158	(null)
2409	2007-06-29T08:53:41.984501Z	direct	167	2	48	154	(null)
2410	2008-05-24T09:19:51.985501Z	direct	168	6	45175	156	(null)
2411	2007-05-24T10:22:10.548639Z	direct	169	8	15760.5	156	(null)
2412	2006-03-29T10:22:09.509801Z	direct	170	9	66816	158	(null)
2413	2008-03-29T12:34:04.525934Z	direct	101	5	48552	161	(null)
2414	2007-03-29T13:22:40.536996Z	direct	102	8	10794.6	153	(null)
2415	2006-03-29T13:34:50.545196Z	direct	103	6	310	161	(null)
2416	2007-03-29T15:41:20.945676Z	direct	104	6	384	160	(null)
2417	2007-03-20T16:49:10.974352Z	direct	105	5	1926.6	163	(null)
2418	2004-03-20T16:18:21.862632Z	direct	106	4	5546.6	163	(null)
2419	2007-03-20T18:03:32.764632Z	direct	107	3	31574	160	(null)
2420	2007-03-13T19:59:43.66632Z	direct	108	2	29750	160	(null)
2421	2007-03-12T20:53:54.562432Z	direct	109	1	72836	(null)	(null)
2422	2007-12-16T20:19:55.462332Z	direct	144	2	11188.5	153	(null)
2423	2007-11-21T10:22:33.362632Z	direct	145	3	10367.7	160	(null)
2424	2007-11-21T10:22:33.263332Z	direct	146	4	13824	153	(null)
2425	2006-11-16T23:34:22.162552Z	direct	147	5	1500.8	163	(null)
2426	2006-11-17T00:22:11.262552Z	direct	148	6	7200	(null)	(null)
2427	2007-11-10T01:34:22.362124Z	direct	149	7	9055	163	(null)
2428	2007-11-10T02:41:34.463567Z	direct	116	8	14685.8	(null)	(null)
2429	2007-11-10T03:49:25.526321Z	direct	117	9	50125	154	(null)
2430	2007-10-02T05:18:36.663332Z	direct	101	8	29669.9	159	(null)
2431	2006-09-14T06:03:04.763452Z	direct	102	1	5610.6	163	(null)
2437	2006-09-01T07:59:15.826132Z	direct	103	4	13550	163	(null)

2438	2007-09-01T08:53:26.934626Z	direct	104	0	5451	154	(null)
2439	2007-08-31T09:19:37.811132Z	direct	105	1	22150.1	159	(null)
2441	2008-08-01T10:22:48.734526Z	direct	106	5	2075.2	160	(null)
2442	2006-07-27T11:22:59.662632Z	direct	107	9	52471.9	154	(null)
2443	2006-07-27T12:34:16.562632Z	direct	108	0	3646	154	(null)
2444	2007-07-27T13:22:27.462632Z	direct	109	1	77727.2	155	(null)
2445	2006-07-27T14:34:38.362632Z	direct	144	8	5537.8	158	(null)
2448	2007-06-18T15:41:49.262632Z	direct	145	5	1388	158	(null)
2449	2007-06-13T16:49:07.162632Z	direct	146	6	86	155	(null)
2450	2007-04-11T17:18:10.362632Z	direct	147	3	1636	159	(null)
2451	2007-12-17T17:03:52.562632Z	direct	148	7	10474.6	154	(null)
2452	2007-10-06T19:59:43.462632Z	direct	149	5	12589	159	(null)
2453	2007-10-04T20:53:34.362632Z	direct	116	0	129	153	(null)
2456	2006-11-07T19:53:25.989889Z	direct	117	0	3878.4	163	(null)
2457	2007-10-31T22:22:16.162632Z	direct	118	5	21586.2	159	(null)

105 rows selected.

2. ORDER_MODE Capitalization (UPPER function)

Write a SQL statement to convert the column ORDER_MODE of your ORDERS_COPY table into Capital letters (use the built-in function UPPER).

```
UPDATE ORDERS_COPY
```

```
SET ORDER_MODE = UPPER(ORDER_MODE);
```

```
9  /* 2. Write a SQL statement to convert the column ORDER_MODE of
10 your ORDERS_COPY table into Capital letters (use the built-in function UPPER). */
11
12 UPDATE ORDERS_COPY
13 SET ORDER_MODE = UPPER(ORDER_MODE);
14
```

Query result **Script output** DBMS output Explain Plan SQL history

Table ORDERS_COPY created.
Elapsed: 00:00:00.045

SQL> UPDATE ORDERS_COPY
 SET ORDER_MODE = UPPER(ORDER_MODE)

105 rows updated.
Elapsed: 00:00:00.003

105 rows updated.

3. Identification of NULL SALES_REP_ID entries

Write a SQL statement to check, by COUNT, if there are NULLs in the SALES_REP_ID column of the table ORDERS_COPY

```
SELECT COUNT(*) AS NULL_SALES_REP_COUNT
```

```
FROM ORDERS_COPY
```

```
WHERE SALES_REP_ID IS NULL;
```

```
1 /
18 SELECT COUNT(*) AS NULL_SALES_REP_COUNT
19 FROM ORDERS_COPY
20 WHERE SALES_REP_ID IS NULL;
21
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.127 seconds

	NULL_SALES_REP_
1	35

```
NULL_SALES_REP_COUNT
-----
```

35

1 rows selected.

4. Row Selection for NULL SALES_REP_ID

Display the rows having null values in the SALES_REP_ID column.

```
SELECT * FROM ORDERS_COPY
WHERE SALES_REP_ID IS NULL;
```

```
22 /* 4. Display the rows having null values in the SALES_REP_ID column. v */
23
24 SELECT * FROM ORDERS_COPY
25 WHERE SALES_REP_ID IS NULL;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.005 seconds

		ORDER_DATE	ORDER_MODE	CUSTOMER_ID	ORDER_STATUS	ORDER_TOTAL	SALES_REP_ID	PROMOTION_ID
1	2355	2006-01-26T09:22:51.962632Z	ONLINE	104	8	94513.5	(null)	(null)
2	2356	2008-01-26T09:22:41.934562Z	ONLINE	105	5	29473.8	(null)	(null)
3	2359	2006-01-08T21:34:13.112233Z	ONLINE	106	9	5543.1	(null)	(null)
4	2360	2007-11-14T12:22:31.223344Z	ONLINE	107	4	990.4	(null)	(null)
5	2361	2007-11-13T13:34:21.98621Z	ONLINE	108	8	120131.3	(null)	(null)
6	2362	2007-11-13T14:41:10.619477Z	ONLINE	109	4	92829.4	(null)	(null)

ORDER_ID	ORDER_DATE	ORDER_MODE	CUSTOMER_ID	ORDER_STATUS	ORDER_TOTAL	SALES_REP_ID	PROMOTION_ID
2355	2006-01-26T09:22:51.962632Z	ONLINE	104	8	94513.5	(null)	(null)
2356	2008-01-26T09:22:41.934562Z	ONLINE	105	5	29473.8	(null)	(null)
2359	2006-01-08T21:34:13.112233Z	ONLINE	106	9	5543.1	(null)	(null)
2360	2007-11-14T12:22:31.223344Z	ONLINE	107	4	990.4	(null)	(null)
2361	2007-11-13T13:34:21.98621Z	ONLINE	108	8	120131.3	(null)	(null)
2362	2007-11-13T14:41:10.619477Z	ONLINE	109	4	92829.4	(null)	(null)
2363	2007-10-23T16:49:56.346122Z	ONLINE	144	0	10082.3	(null)	(null)
2364	2007-08-28T17:18:45.942399Z	ONLINE	145	4	9500	(null)	(null)
2365	2007-08-28T18:03:34.003399Z	ONLINE	146	9	27455.3	(null)	(null)
2366	2007-08-28T19:59:23.144778Z	ONLINE	147	5	37319.4	(null)	(null)
2367	2008-06-27T20:53:32.335522Z	ONLINE	148	10	144054.8	(null)	(null)
2368	2008-06-26T21:19:43.190089Z	ONLINE	149	10	60065	(null)	(null)
2369	2007-06-26T22:22:54.009932Z	ONLINE	116	0	11097.4	(null)	(null)
2370	2008-06-26T23:22:11.647398Z	ONLINE	117	4	126	(null)	(null)

2371	2007-05-16T00:34:56.113356Z	ONLINE	118	6	79405.6	(null)	(null)
2372	2007-02-27T00:22:33.356789Z	ONLINE	119	9	16447.2	(null)	(null)
2373	2008-02-27T01:34:51.220065Z	ONLINE	120	4	416	(null)	(null)
2374	2008-02-27T02:41:45.109654Z	ONLINE	121	0	4797	(null)	(null)
2375	2007-02-26T03:49:50.459233Z	ONLINE	122	2	103834.4	(null)	(null)
2376	2007-06-07T05:18:08.88331Z	ONLINE	123	6	11006.2	(null)	(null)
2377	2007-06-07T06:03:01.0011Z	ONLINE	141	5	38017.8	(null)	(null)
2378	2007-05-24T07:59:10.010101Z	ONLINE	142	5	25691.3	(null)	(null)
2380	2007-05-16T08:53:02.90909Z	ONLINE	143	3	27132.6	(null)	(null)
2382	2008-05-14T09:19:03.828321Z	ONLINE	144	8	71173	(null)	(null)
2383	2008-05-12T10:22:30.545103Z	ONLINE	145	8	36374.7	(null)	(null)
2384	2008-05-12T11:22:34.525972Z	ONLINE	146	3	29249.1	(null)	(null)
2385	2007-12-08T11:34:11.331392Z	ONLINE	147	4	295892	(null)	(null)
2386	2007-12-06T12:22:34.225609Z	ONLINE	148	10	21116.9	(null)	(null)
2387	2007-03-11T14:34:56.536966Z	ONLINE	149	5	52758.9	(null)	(null)
2388	2007-06-04T15:41:12.554435Z	ONLINE	150	4	282694.3	(null)	(null)
2389	2008-06-04T16:49:43.546954Z	ONLINE	151	4	17620	(null)	(null)
2390	2007-11-18T16:18:50.546851Z	ONLINE	152	9	7616.8	(null)	(null)
2421	2007-03-12T20:53:54.562432Z	DIRECT	109	1	72836	(null)	(null)
2426	2006-11-17T00:22:11.262552Z	DIRECT	148	6	7200	(null)	(null)
2428	2007-11-10T02:41:34.463567Z	DIRECT	116	8	14685.8	(null)	(null)

35 rows selected.

5. Replacement of NULL IDs with 999 using NVL

Write a SQL statement to replace all NULLs in the column SALES_REP_ID of the table ORDERS_COPY with 999 (as an ID for a virtual Sales Representative).

```
UPDATE ORDERS_COPY
```

```
SET SALES_REP_ID = NVL(SALES_REP_ID, 999);
```

```
27 /*5. Write a SQL statement to replace all NULLs in the column SALES_REP_ID of the table
28 ORDERS_COPY with 999 (as an ID for a virtual Sales Representative). */
29
30 UPDATE ORDERS_COPY
31 SET SALES_REP_ID = NVL(SALES_REP_ID, 999);
32
```

Query result Script output DBMS output Explain Plan SQL history



```
SQL> UPDATE ORDERS_COPY
      SET SALES_REP_ID = NVL(SALES_REP_ID, 999)
```

105 rows updated.

Elapsed: 00:00:00.004

105 rows updated.

6. Content Verification

Display the content of the table ORDERS_COPY

```
SELECT * FROM ORDERS_COPY;
```

```
33 /* 6. Display the content of the table ORDERS_COPY. */
34
35 SELECT * FROM ORDERS_COPY;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.011 seconds

	ORDER_ID	ORDER_DATE	ORDER_MODE	CUSTOMER_ID	ORDER_STATUS	ORDER_TOTAL	SALES_REP_ID	PROMC
1	2458	2007-08-16T14:34:12.234359Z	DIRECT	101	0	78279.6	153	
2	2397	2007-11-19T14:41:58.000000Z	DIRECT	102	1	42283.2	154	
3	2454	2007-10-02T16:49:00.000000Z	DIRECT	103	1	6653.4	154	
4	2354	2008-07-14T17:18:00.000000Z	DIRECT	104	0	46257	155	
5	2358	2008-01-08T17:03:00.000000Z	DIRECT	105	2	7826	155	
6	2381	2008-05-14T19:59:00.000000Z	DIRECT	106	3	23034.6	156	

ORDER_ID	ORDER_DATE	ORDER_MODE	CUSTOMER_ID	ORDER_STATUS	ORDER_TOTAL	SALES_REP_ID	PROMOTION_ID
2458	2007-08-16T14:34:12.234359Z	DIRECT	101	0	78279.6	153	(null)

2397	2007-11-19T14:41:54.696211Z	DIRECT	102	1	42283.2	154	(null)
2454	2007-10-02T16:49:34.67834Z	DIRECT	103	1	6653.4	154	(null)
2354	2008-07-14T17:18:23.234567Z	DIRECT	104	0	46257	155	(null)
2358	2008-01-08T17:03:12.654278Z	DIRECT	105	2	7826	155	(null)
2381	2008-05-14T19:59:08.843679Z	DIRECT	106	3	23034.6	156	(null)
2440	2007-08-31T20:53:06.008765Z	DIRECT	107	3	70576.9	156	(null)
2357	2006-01-08T20:19:44.123456Z	DIRECT	108	5	59872.4	158	(null)
2394	2008-02-10T21:22:35.564789Z	DIRECT	109	5	21863	158	(null)
2435	2007-09-02T22:22:53.134567Z	DIRECT	144	6	62303	159	(null)
2455	2007-09-20T10:34:11.456789Z	DIRECT	145	7	14087.5	160	(null)
2379	2007-05-16T01:22:24.234567Z	DIRECT	146	8	17848.2	161	(null)
2396	2006-02-02T01:34:56.345678Z	DIRECT	147	8	34930	161	(null)
2406	2007-06-29T03:41:20.098765Z	DIRECT	148	8	2854.2	161	(null)
2434	2007-09-13T04:49:30.647893Z	DIRECT	149	8	268651.8	161	(null)
2436	2007-09-02T05:18:04.378034Z	DIRECT	116	8	6394.8	161	(null)
2446	2007-07-27T06:03:08.302945Z	DIRECT	117	8	103679.3	161	(null)
2447	2008-07-27T07:59:10.223344Z	DIRECT	101	8	33893.6	161	(null)
2432	2007-09-14T08:53:40.223345Z	DIRECT	102	10	10523	163	(null)
2433	2007-09-13T09:19:00.654279Z	DIRECT	103	10	78	163	(null)
2355	2006-01-26T09:22:51.962632Z	ONLINE	104	8	94513.5	999	(null)
2356	2008-01-26T09:22:41.934562Z	ONLINE	105	5	29473.8	999	(null)
2359	2006-01-08T21:34:13.112233Z	ONLINE	106	9	5543.1	999	(null)
2360	2007-11-14T12:22:31.223344Z	ONLINE	107	4	990.4	999	(null)
2361	2007-11-13T13:34:21.98621Z	ONLINE	108	8	120131.3	999	(null)
2362	2007-11-13T14:41:10.619477Z	ONLINE	109	4	92829.4	999	(null)
2363	2007-10-23T16:49:56.346122Z	ONLINE	144	0	10082.3	999	(null)

2364	2007-08-28T17:18:45.942399Z	ONLINE	145	4	9500	999	(null)
2365	2007-08-28T18:03:34.003399Z	ONLINE	146	9	27455.3	999	(null)
2366	2007-08-28T19:59:23.144778Z	ONLINE	147	5	37319.4	999	(null)
2367	2008-06-27T20:53:32.335522Z	ONLINE	148	10	144054.8	999	(null)
2368	2008-06-26T21:19:43.190089Z	ONLINE	149	10	60065	999	(null)
2369	2007-06-26T22:22:54.009932Z	ONLINE	116	0	11097.4	999	(null)
2370	2008-06-26T23:22:11.647398Z	ONLINE	117	4	126	999	(null)
2371	2007-05-16T00:34:56.113356Z	ONLINE	118	6	79405.6	999	(null)
2372	2007-02-27T00:22:33.356789Z	ONLINE	119	9	16447.2	999	(null)
2373	2008-02-27T01:34:51.220065Z	ONLINE	120	4	416	999	(null)
2374	2008-02-27T02:41:45.109654Z	ONLINE	121	0	4797	999	(null)
2375	2007-02-26T03:49:50.459233Z	ONLINE	122	2	103834.4	999	(null)
2376	2007-06-07T05:18:08.88331Z	ONLINE	123	6	11006.2	999	(null)
2377	2007-06-07T06:03:01.0011Z	ONLINE	141	5	38017.8	999	(null)
2378	2007-05-24T07:59:10.010101Z	ONLINE	142	5	25691.3	999	(null)
2380	2007-05-16T08:53:02.90909Z	ONLINE	143	3	27132.6	999	(null)
2382	2008-05-14T09:19:03.828321Z	ONLINE	144	8	71173	999	(null)
2383	2008-05-12T10:22:30.545103Z	ONLINE	145	8	36374.7	999	(null)
2384	2008-05-12T11:22:34.525972Z	ONLINE	146	3	29249.1	999	(null)
2385	2007-12-08T11:34:11.331392Z	ONLINE	147	4	295892	999	(null)
2386	2007-12-06T12:22:34.225609Z	ONLINE	148	10	21116.9	999	(null)
2387	2007-03-11T14:34:56.536966Z	ONLINE	149	5	52758.9	999	(null)
2388	2007-06-04T15:41:12.554435Z	ONLINE	150	4	282694.3	999	(null)
2389	2008-06-04T16:49:43.546954Z	ONLINE	151	4	17620	999	(null)
2390	2007-11-18T16:18:50.546851Z	ONLINE	152	9	7616.8	999	(null)
2391	2006-02-27T17:03:03.82833Z	DIRECT	153	2	48070.6	156	(null)

2392	2007-07-21T19:59:57.571057Z	DIRECT	154	9	26632	161	(null)
2393	2008-02-10T19:53:19.528202Z	DIRECT	155	4	23431.9	161	(null)
2395	2006-02-02T20:19:11.22755Z	DIRECT	156	3	68501	163	(null)
2398	2007-11-19T21:22:53.224175Z	DIRECT	157	9	7110.3	163	(null)
2399	2007-11-19T22:22:38.34099Z	DIRECT	158	0	25270.3	161	(null)
2400	2007-07-10T00:34:29.559387Z	DIRECT	159	2	69286.4	161	(null)
2401	2007-07-10T01:22:53.554822Z	DIRECT	160	3	969.2	163	(null)
2402	2007-07-02T02:34:44.66517Z	DIRECT	161	8	600	154	(null)
2403	2007-07-01T15:49:13.615512Z	DIRECT	162	0	220	154	(null)
2404	2007-07-01T15:49:13.664085Z	DIRECT	163	6	510	158	(null)
2405	2007-07-01T15:49:13.678123Z	DIRECT	164	5	1233	159	(null)
2407	2007-06-29T06:03:21.526005Z	DIRECT	165	9	2519	155	(null)
2408	2007-06-29T07:59:31.333617Z	DIRECT	166	1	309	158	(null)
2409	2007-06-29T08:53:41.984501Z	DIRECT	167	2	48	154	(null)
2410	2008-05-24T09:19:51.985501Z	DIRECT	168	6	45175	156	(null)
2411	2007-05-24T10:22:10.548639Z	DIRECT	169	8	15760.5	156	(null)
2412	2006-03-29T10:22:09.509801Z	DIRECT	170	9	66816	158	(null)
2413	2008-03-29T12:34:04.525934Z	DIRECT	101	5	48552	161	(null)
2414	2007-03-29T13:22:40.536996Z	DIRECT	102	8	10794.6	153	(null)
2415	2006-03-29T13:34:50.545196Z	DIRECT	103	6	310	161	(null)
2416	2007-03-29T15:41:20.945676Z	DIRECT	104	6	384	160	(null)
2417	2007-03-20T16:49:10.974352Z	DIRECT	105	5	1926.6	163	(null)
2418	2004-03-20T16:18:21.862632Z	DIRECT	106	4	5546.6	163	(null)
2419	2007-03-20T18:03:32.764632Z	DIRECT	107	3	31574	160	(null)
2420	2007-03-13T19:59:43.66632Z	DIRECT	108	2	29750	160	(null)
2421	2007-03-12T20:53:54.562432Z	DIRECT	109	1	72836	999	(null)

2422	2007-12-16T20:19:55.462332Z	DIRECT	144	2	11188.5	153	(null)
2423	2007-11-21T10:22:33.362632Z	DIRECT	145	3	10367.7	160	(null)
2424	2007-11-21T10:22:33.263332Z	DIRECT	146	4	13824	153	(null)
2425	2006-11-16T23:34:22.162552Z	DIRECT	147	5	1500.8	163	(null)
2426	2006-11-17T00:22:11.262552Z	DIRECT	148	6	7200	999	(null)
2427	2007-11-10T01:34:22.362124Z	DIRECT	149	7	9055	163	(null)
2428	2007-11-10T02:41:34.463567Z	DIRECT	116	8	14685.8	999	(null)
2429	2007-11-10T03:49:25.526321Z	DIRECT	117	9	50125	154	(null)
2430	2007-10-02T05:18:36.663332Z	DIRECT	101	8	29669.9	159	(null)
2431	2006-09-14T06:03:04.763452Z	DIRECT	102	1	5610.6	163	(null)
2437	2006-09-01T07:59:15.826132Z	DIRECT	103	4	13550	163	(null)
2438	2007-09-01T08:53:26.934626Z	DIRECT	104	0	5451	154	(null)
2439	2007-08-31T09:19:37.811132Z	DIRECT	105	1	22150.1	159	(null)
2441	2008-08-01T10:22:48.734526Z	DIRECT	106	5	2075.2	160	(null)
2442	2006-07-27T11:22:59.662632Z	DIRECT	107	9	52471.9	154	(null)
2443	2006-07-27T12:34:16.562632Z	DIRECT	108	0	3646	154	(null)
2444	2007-07-27T13:22:27.462632Z	DIRECT	109	1	77727.2	155	(null)
2445	2006-07-27T14:34:38.362632Z	DIRECT	144	8	5537.8	158	(null)
2448	2007-06-18T15:41:49.262632Z	DIRECT	145	5	1388	158	(null)
2449	2007-06-13T16:49:07.162632Z	DIRECT	146	6	86	155	(null)
2450	2007-04-11T17:18:10.362632Z	DIRECT	147	3	1636	159	(null)
2451	2007-12-17T17:03:52.562632Z	DIRECT	148	7	10474.6	154	(null)
2452	2007-10-06T19:59:43.462632Z	DIRECT	149	5	12589	159	(null)
2453	2007-10-04T20:53:34.362632Z	DIRECT	116	0	129	153	(null)
2456	2006-11-07T19:53:25.989889Z	DIRECT	117	0	3878.4	163	(null)
2457	2007-10-31T22:22:16.162632Z	DIRECT	118	5	21586.2	159	(null)

105 rows selected.

7. PROMOTION_ID Count Analysis

Run the statement `SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;` Note that you get 0 although there are 105 rows with NULL in the column `PROMOTION_ID`.

```
SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;
```

```
36
37 /* 7. Run the statement SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY; Note that you get
38 0 although there are 105 rows with NULL in the column PROMOTION_ID. */
39
40 SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;
```

Query result		Script output	DBMS output	Explain Plan	SQL history
		Download Execution time: 0.004 seconds			
		COUNT(PROMOTION_ID)			
1		0			

COUNT(PROMOTION_ID)

0

1 rows selected.

8. Theoretical Analysis of COUNT behavior

Explain why you get zero

The reason you see 0 instead of 105 is due to how SQL functions work:

- **The COUNT Rule:** When you use `COUNT(column_name)`, SQL only counts rows that have **actual data** in them.
- **Ignoring NULLs:** This specific function is designed to **ignore** or skip all NULL (empty) values.
- **Your Table's Data:** Since all 105 rows in the `PROMOTION_ID` column are currently NULL, the function finds nothing to count, resulting in 0.

9. Replacement of NULL PROMOTION_ID with 0 using NVL

Write a SQL statement to replace all nulls in the PROMOTION_ID column of the table ORDERS_COPY with 0 (zero).

```
UPDATE ORDERS_COPY
```

```
SET PROMOTION_ID = NVL(PROMOTION_ID, 0);
```

```
42  /* 9. Write a SQL statement to replace all nulls in the PROMOTION_ID column of the table
43  ORDERS_COPY with 0 (zero).*/
44
45  UPDATE ORDERS_COPY
46  SET PROMOTION_ID = NVL(PROMOTION_ID, 0);
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> UPDATE ORDERS_COPY
      SET PROMOTION_ID = NVL(PROMOTION_ID, 0)
```

105 rows updated.

Elapsed: 00:00:00.004

105 rows updated.

10. Count Verification and Value Explanation

Re-run the statement `SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;` Explain why this result value.

```
SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;
```

```
48  /* 10. Re-run the statement SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY; Explain why
49  this result value. */
50
51  SELECT COUNT(PROMOTION_ID) FROM ORDERS_COPY;
```

Query result		Script output	DBMS output	Explain Plan	SQL history
Download		Execution time: 0.003 seconds			
COUNT(PROMOTION_ID)					
1	105				

COUNT(PROMOTION_ID)

105

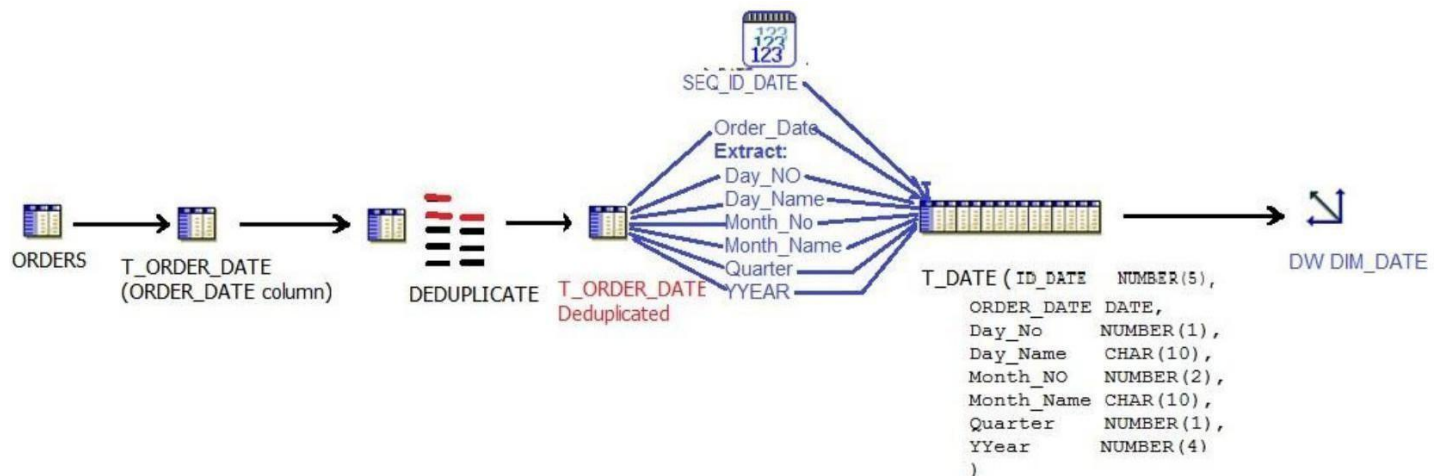
1 rows selected.

Explanation of the Result

- **From NULL to Zero:** In the previous step (Question 9), you executed an UPDATE statement that replaced every NULL in the PROMOTION_ID column with the number 0.
- **0 is a Value:** While SQL's COUNT(column) function ignores NULL because it represents an absence of data, it **does count** the number 0 because it is a valid, non-null numeric value.
- **Full Count:** Because the column now contains 105 rows of data (the number 0) instead of "nothing," the function sees 105 valid entries to count.

In short: The COUNT function now sees **data** where it previously saw **emptiness**.

II. Create T_DATE Table



1. T_ORDER_DATE Staging Table Creation

Use the CREATE and SELECT statements to create one-column temporary table T_ORDER_DATE (ORDER_DATE DATE) containing ORDER_DATE values (formatted as DD-MM-YYYY) from the OE.ORDERS table.

```
CREATE TABLE T_ORDER_DATE AS
SELECT TO_CHAR(ORDER_DATE, 'DD-MM-YYYY') AS ORDER_DATE
FROM OE.ORDERS;
```

53 /* 1. Use the CREATE and SELECT statements to create one-column temporary table
54 T_ORDER_DATE (ORDER_DATE DATE) containing ORDER_DATE values (formatted as
55 DD-MM-YYYY) from the OE.ORDERS table. */
56
57 CREATE TABLE T_ORDER_DATE AS
58 SELECT TO_CHAR(ORDER_DATE, 'DD-MM-YYYY') AS ORDER_DATE
59 FROM OE.ORDERS;

Query result Script output DBMS output Explain Plan SQL history



```
SQL> CREATE TABLE T_ORDER_DATE AS
      SELECT TO_CHAR(ORDER_DATE, 'DD-MM-YYYY') AS ORDER_DATE
      FROM OE.ORDERS
```

Table T_ORDER_DATE created.

Elapsed: 00:00:00.036

Table T_ORDER_DATE created.

2. ROWID and Content Display

Display the whole content of T_ORDER_DATE including the ROWID pseudo-column. Note that ROWID values are all distinct (even for identical rows).

```
SELECT ORDER_DATE, ROWID
```

```
FROM T_ORDER_DATE;
```

```
61  /* 2. Display the whole content of T_ORDER_DATE including the ROWID pseudo-column.
62  Note that ROWID values are all distinct (even for identical rows). */
63
64  SELECT ORDER_DATE, ROWID
65  FROM T_ORDER_DATE;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.006 seconds

	ORDER_DATE	ROWID
1	16-08-2007	AANsr8AACAABQm7AAA
2	19-11-2007	AANsr8AACAABQm7AAB
3	02-10-2007	AANsr8AACAABQm7AAC
4	14-07-2008	AANsr8AACAABQm7AAD
5	08-01-2008	AANsr8AACAABQm7AAE
6	14-05-2008	AANsr8AACAABQm7AAF

ORDER_DATE	ROWID
16-08-2007	AANvFjAABAAP0ijAAA
19-11-2007	AANvFjAABAAP0ijAAB
2-10-2007	AANvFjAABAAP0ijAAC
14-07-2008	AANvFjAABAAP0ijAAD
8-1-2008	AANvFjAABAAP0ijAAE
14-05-2008	AANvFjAABAAP0ijAAF
31-08-2007	AANvFjAABAAP0ijAAG
8-1-2006	AANvFjAABAAP0ijAAH
10-2-2008	AANvFjAABAAP0ijAAI
2-9-2007	AANvFjAABAAP0ijAAJ
20-09-2007	AANvFjAABAAP0ijAAK
16-05-2007	AANvFjAABAAP0ijAAL
2-2-2006	AANvFjAABAAP0ijAAM
29-06-2007	AANvFjAABAAP0ijAAN
13-09-2007	AANvFjAABAAP0ijAAO
2-9-2007	AANvFjAABAAP0ijAAP
27-07-2007	AANvFjAABAAP0ijAAQ
27-07-2008	AANvFjAABAAP0ijAAR
14-09-2007	AANvFjAABAAP0ijAAS
13-09-2007	AANvFjAABAAP0ijAAT
26-01-2006	AANvFjAABAAP0ijAAU
26-01-2008	AANvFjAABAAP0ijAAV
8-1-2006	AANvFjAABAAP0ijAAW

14-11-2007	AANvFjAABAAP0ijAAX
13-11-2007	AANvFjAABAAP0ijAAY
13-11-2007	AANvFjAABAAP0ijAAZ
23-10-2007	AANvFjAABAAP0ijAAa
28-08-2007	AANvFjAABAAP0ijAAb
28-08-2007	AANvFjAABAAP0ijAAc
28-08-2007	AANvFjAABAAP0ijAAd
27-06-2008	AANvFjAABAAP0ijAAe
26-06-2008	AANvFjAABAAP0ijAAf
26-06-2007	AANvFjAABAAP0ijAAg
26-06-2008	AANvFjAABAAP0ijAAh
16-05-2007	AANvFjAABAAP0ijAAi
27-02-2007	AANvFjAABAAP0ijAAj
27-02-2008	AANvFjAABAAP0ijAAk
27-02-2008	AANvFjAABAAP0ijAAl
26-02-2007	AANvFjAABAAP0ijAAm
7-6-2007	AANvFjAABAAP0ijAAn
7-6-2007	AANvFjAABAAP0ijAAo
24-05-2007	AANvFjAABAAP0ijAAp
16-05-2007	AANvFjAABAAP0ijAAq
14-05-2008	AANvFjAABAAP0ijAAr
12-5-2008	AANvFjAABAAP0ijAAs
12-5-2008	AANvFjAABAAP0ijAAt
8-12-2007	AANvFjAABAAP0ijAAu
6-12-2007	AANvFjAABAAP0ijAAv
11-3-2007	AANvFjAABAAP0ijAAw
4-6-2007	AANvFjAABAAP0ijAAx
4-6-2008	AANvFjAABAAP0ijAAy
18-11-2007	AANvFjAABAAP0ijAAz
27-02-2006	AANvFjAABAAP0ijAA0
21-07-2007	AANvFjAABAAP0ijAA1
10-2-2008	AANvFjAABAAP0ijAA2
2-2-2006	AANvFjAABAAP0ijAA3
19-11-2007	AANvFjAABAAP0ijAA4
19-11-2007	AANvFjAABAAP0ijAA5
10-7-2007	AANvFjAABAAP0ijAA6
10-7-2007	AANvFjAABAAP0ijAA7
2-7-2007	AANvFjAABAAP0ijAA8
1-7-2007	AANvFjAABAAP0ijAA9
1-7-2007	AANvFjAABAAP0ijAA+
1-7-2007	AANvFjAABAAP0ijAA/
29-06-2007	AANvFjAABAAP0ijABA
29-06-2007	AANvFjAABAAP0ijABB
29-06-2007	AANvFjAABAAP0ijABC
24-05-2008	AANvFjAABAAP0ijABD
24-05-2007	AANvFjAABAAP0ijABE

29-03-2006	AANvFjAABAAP0ijABF
29-03-2008	AANvFjAABAAP0ijABG
29-03-2007	AANvFjAABAAP0ijABH
29-03-2006	AANvFjAABAAP0ijABI
29-03-2007	AANvFjAABAAP0ijABJ
20-03-2007	AANvFjAABAAP0ijABK
20-03-2004	AANvFjAABAAP0ijABL
20-03-2007	AANvFjAABAAP0ijABM
13-03-2007	AANvFjAABAAP0ijABN
12-3-2007	AANvFjAABAAP0ijABO
16-12-2007	AANvFjAABAAP0ijABP
21-11-2007	AANvFjAABAAP0ijABQ
21-11-2007	AANvFjAABAAP0ijABR
16-11-2006	AANvFjAABAAP0ijABS
17-11-2006	AANvFjAABAAP0ijABT
10-11-2007	AANvFjAABAAP0ijABU
10-11-2007	AANvFjAABAAP0ijABV
10-11-2007	AANvFjAABAAP0ijABW
2-10-2007	AANvFjAABAAP0ijABX
14-09-2006	AANvFjAABAAP0ijABY
1-9-2006	AANvFjAABAAP0ijABZ
1-9-2007	AANvFjAABAAP0ijABa
31-08-2007	AANvFjAABAAP0ijABb
1-8-2008	AANvFjAABAAP0ijABc
27-07-2006	AANvFjAABAAP0ijABd
27-07-2006	AANvFjAABAAP0ijABe
27-07-2007	AANvFjAABAAP0ijABf
27-07-2006	AANvFjAABAAP0ijABg
18-06-2007	AANvFjAABAAP0ijABh
13-06-2007	AANvFjAABAAP0ijABi
11-4-2007	AANvFjAABAAP0ijABj
17-12-2007	AANvFjAABAAP0ijABk
6-10-2007	AANvFjAABAAP0ijABl
4-10-2007	AANvFjAABAAP0ijABm
7-11-2006	AANvFjAABAAP0ijABn
31-10-2007	AANvFjAABAAP0ijABo

70 rows selected.

3. Distinct Date Count and Frequency Analysis

Check the duplicates using one of the following 2 methods:

a) Write a *SELECT* to display the number of distinct values (You should get 70).

```
SELECT COUNT(DISTINCT ORDER_DATE) AS Distinct_Dates
FROM T_ORDER_DATE;
```

```
67  /* 3. Check the duplicates using one of the following 2 methods:
68  a) Write a SELECT to display the number of distinct values (You should get 70). */
69
70  SELECT COUNT(DISTINCT ORDER_DATE) AS Distinct_Dates
71  FROM T_ORDER_DATE;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.003 seconds

	DISTINCT_DATES
1	70

DISTINCT_DATES

70

1 rows selected.

b) Display the number of occurrences of each *ORDER_DATE* value.

```
SELECT ORDER_DATE, COUNT(*) AS Occurrences
FROM T_ORDER_DATE
GROUP BY ORDER_DATE;
```

```
73  /* b) Display the number of occurrences of each ORDER_DATE value. */
74
75  SELECT ORDER_DATE, COUNT(*) AS Occurrences
76  FROM T_ORDER_DATE
77  GROUP BY ORDER_DATE;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.009 seconds

	ORDER_DATE	OCCURRENCES
1	16-08-2007	1
2	19-11-2007	3
3	02-10-2007	2
4	14-07-2008	1
5	08-01-2008	1
6	14-05-2008	2
7	31-08-2007	2

ORDER_DATE	OCCURRENCES
16-08-2007	1
19-11-2007	3
2-10-2007	2
14-07-2008	1
8-1-2008	1

14-05-2008	2
31-08-2007	2
8-1-2006	2
10-2-2008	2
2-9-2007	2
20-09-2007	1
16-05-2007	3
2-2-2006	2
29-06-2007	4
13-09-2007	2
27-07-2007	2
27-07-2008	1
14-09-2007	1
26-01-2006	1
26-01-2008	1
14-11-2007	1
13-11-2007	2
23-10-2007	1
28-08-2007	3
27-06-2008	1
26-06-2008	2
26-06-2007	1
27-02-2007	1
27-02-2008	2
26-02-2007	1
7-6-2007	2
24-05-2007	2
12-5-2008	2
8-12-2007	1
6-12-2007	1
11-3-2007	1
4-6-2007	1
4-6-2008	1
18-11-2007	1
27-02-2006	1
21-07-2007	1
10-7-2007	2
2-7-2007	1
1-7-2007	3
24-05-2008	1
29-03-2006	2
29-03-2008	1
29-03-2007	2
20-03-2007	2
20-03-2004	1
13-03-2007	1

12-3-2007	1
16-12-2007	1
21-11-2007	2
16-11-2006	1
17-11-2006	1
10-11-2007	3
14-09-2006	1
1-9-2006	1
1-9-2007	1
1-8-2008	1
27-07-2006	3
18-06-2007	1
13-06-2007	1
11-4-2007	1
17-12-2007	1
6-10-2007	1
4-10-2007	1
7-11-2006	1
31-10-2007	1

70 rows selected.

4. Deduplication Methodology

With this data above, it requires deduplication. Deduplication is a frequent ETL task to prepare Non-Redundant data for loading dimensions and/or facts. It keeps a single value of each repeated value.

The **T_ORDER_DATE** table could be created (from the beginning) without redundant rows in order to avoid deduplicating it.

a) DROP T_ORDER_DATE.

DROP TABLE T_ORDER_DATE;

The screenshot shows a SQL IDE interface. At the top, there is a code editor with the following SQL script:

```

79  /* question 4. a) DROP T_ORDER_DATE. */
80
81  DROP TABLE T_ORDER_DATE;

```

Below the code editor, there is a tabbed interface with the following tabs: "Query result", "Script output", "DBMS output", "Explain Plan", and "SQL history". The "Script output" tab is currently selected. Below the tabs, there is a toolbar with icons for "Copy" and "Download". Below the toolbar, there is a text area showing the execution results:

```

SQL> DROP TABLE T_ORDER_DATE

```

Below the text area, there is a message: "Table T_ORDER_DATE dropped." and a timestamp: "Elapsed: 00:00:00.024".

Table T_ORDER_DATE dropped.

b) Rewrite your answer to question 1 to obtain directly distinct values of dates in T_ORDER_DATE

```
CREATE TABLE T_ORDER_DATE AS  
SELECT DISTINCT TO_DATE(TO_CHAR(ORDER_DATE, 'DD-MM-YYYY'), 'DD-MM-YYYY') AS ORDER_DATE  
FROM OE.ORDERS;
```

```
83 /* b) Rewrite your answer to question 1 to obtain directly distinct values of dates in  
84 T_ORDER_DATE */  
85  
86 CREATE TABLE T_ORDER_DATE AS  
87 SELECT DISTINCT TO_DATE(TO_CHAR(ORDER_DATE, 'DD-MM-YYYY'), 'DD-MM-YYYY') AS ORDER_DATE  
88 FROM OE.ORDERS;
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> CREATE TABLE T_ORDER_DATE AS  
      SELECT DISTINCT TO_DATE(TO_CHAR(ORDER_DATE, 'DD-MM-YYYY'), 'DD-MM-YYYY') AS ORDER_DATE  
      FROM OE.ORDERS
```

Table T_ORDER_DATE created.

Elapsed: 00:00:00.035

Table T_ORDER_DATE created.

5. T_DATE Table Structure Definition

Whatever the method you have used so far, we have deduplicated table **T_ORDER_DATE**. Create a temporary table (assumed in the staging area) called **T_DATE**.

```
CREATE TABLE T_DATE (  
    ORDER_DATE DATE,  
    Day_No NUMBER(1),  
    Day_Name CHAR(10),  
    Month_No NUMBER(2),  
    Month_Name CHAR(10),  
    Quarter NUMBER(1),  
    YYear NUMBER(4)  
);  
  
90  /* 5. Whatever the method you have used so far, you have deduplicated table  
91  T_ORDER_DATE. Create a temporary table (assumed in the staging area) called T_DATE. */  
92  
93  CREATE TABLE T_DATE (  
94      ORDER_DATE DATE,  
95      Day_No NUMBER(1),  
96      Day_Name CHAR(10),  
97      Month_No NUMBER(2),  
98      Month_Name CHAR(10),  
99      Quarter NUMBER(1),  
100     YYear NUMBER(4)  
101 );
```

Query result **Script output** DBMS output Explain Plan SQL history



Table T_ORDER_DATE created.

Elapsed: 00:00:00.035

SQL> CREATE TABLE T_DATE (
 ORDER_DATE DATE,
 Day_No NUMBER(1),
 Day_Name CHAR(10),...

Show more...

Table T_DATE created.

Elapsed: 00:00:00.012

Table T_DATE created.

6. Population via Extraction Functions (TO_CHAR/TO_NUMBER)

Populate the T_DATE table from T_ORDER_DATE (use the built-in functions TO_CHAR for extracting the Date elements and TO_NUMBER to convert into NUMBER).

```
INSERT INTO T_DATE (ORDER_DATE, Day_No, Day_Name, Month_No, Month_Name, Quarter, YYear)
SELECT
```

```
ORDER_DATE,
TO_NUMBER(TO_CHAR(ORDER_DATE, 'D')),
TO_CHAR(ORDER_DATE, 'DAY'),
TO_NUMBER(TO_CHAR(ORDER_DATE, 'MM')),
TO_CHAR(ORDER_DATE, 'MONTH'),
TO_NUMBER(TO_CHAR(ORDER_DATE, 'Q')),
TO_NUMBER(TO_CHAR(ORDER_DATE, 'YYYY'))
```

```
FROM T_ORDER_DATE;
```

```
SELECT * FROM T_DATE;
```

```
103 INSERT INTO T_DATE (ORDER_DATE, DAY_NO, DAY_NAME, MONTH_NO, MONTH_NAME, QUARTER, YYEAR)
104 SELECT
105     ORDER_DATE,
106     TO_NUMBER(TO_CHAR(ORDER_DATE, 'D')), -- Day number (1-7)
107     TO_CHAR(ORDER_DATE, 'DAY'), -- Name of the day
108     TO_NUMBER(TO_CHAR(ORDER_DATE, 'MM')), -- Month number (01-12)
109     TO_CHAR(ORDER_DATE, 'MONTH'), -- Name of the month
110     TO_NUMBER(TO_CHAR(ORDER_DATE, 'Q')), -- Quarter (1-4)
111     TO_NUMBER(TO_CHAR(ORDER_DATE, 'YYYY')) -- 4-digit Year
112 FROM T_ORDER_DATE;
113
114 /* 6. Populate the T_DATE table from T_ORDER_DATE (use the built-in functions
115 TO_CHAR for extracting the Date elements and TO_NUMBER to convert into
116 NUMBER). */
117
118 SELECT * FROM T_DATE;
```



Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.01 seconds

	ORDER_DATE	DAY_NO	DAY_NAME	MONTH_NO	MONTH_NAME	QUARTER	YYEAR
1	8/16/2007, 12:00:00	5	THURSDAY	8	AUGUST	3	2007
2	11/19/2007, 12:00:00	2	MONDAY	11	NOVEMBER	4	2007
3	10/2/2007, 12:00:00	3	TUESDAY	10	OCTOBER	4	2007
4	7/14/2008, 12:00:00	2	MONDAY	7	JULY	3	2008
5	1/8/2008, 12:00:00	3	TUESDAY	1	JANUARY	1	2008
6	5/14/2008, 12:00:00	4	WEDNESDAY	5	MAY	2	2008
7	8/31/2007, 12:00:00	6	FRIDAY	8	AUGUST	3	2007

70 rows inserted.

ORDER_DATE	DAY_NO	DAY_NAME	MONTH_NO	MONTH_NAME	QUARTER	YYEAR
2007-08-16T00:00:00Z	5	THURSDAY	8	AUGUST	3	2007
2007-11-19T00:00:00Z	2	MONDAY	11	NOVEMBER	4	2007
2007-10-02T00:00:00Z	3	TUESDAY	10	OCTOBER	4	2007
2008-07-14T00:00:00Z	2	MONDAY	7	JULY	3	2008
2008-01-08T00:00:00Z	3	TUESDAY	1	JANUARY	1	2008
2008-05-14T00:00:00Z	4	WEDNESDAY	5	MAY	2	2008
2007-08-31T00:00:00Z	6	FRIDAY	8	AUGUST	3	2007
2006-01-08T00:00:00Z	1	SUNDAY	1	JANUARY	1	2006
2008-02-10T00:00:00Z	1	SUNDAY	2	FEBRUARY	1	2008
2007-09-02T00:00:00Z	1	SUNDAY	9	SEPTEMBER	3	2007

2007-09-20T00:00:00Z	5	THURSDAY	9	SEPTEMBER	3	2007
2007-05-16T00:00:00Z	4	WEDNESDAY	5	MAY	2	2007
2006-02-02T00:00:00Z	5	THURSDAY	2	FEBRUARY	1	2006
2007-06-29T00:00:00Z	6	FRIDAY	6	JUNE	2	2007
2007-09-13T00:00:00Z	5	THURSDAY	9	SEPTEMBER	3	2007
2007-07-27T00:00:00Z	6	FRIDAY	7	JULY	3	2007
2008-07-27T00:00:00Z	1	SUNDAY	7	JULY	3	2008
2007-09-14T00:00:00Z	6	FRIDAY	9	SEPTEMBER	3	2007
2006-01-26T00:00:00Z	5	THURSDAY	1	JANUARY	1	2006
2008-01-26T00:00:00Z	7	SATURDAY	1	JANUARY	1	2008
2007-11-14T00:00:00Z	4	WEDNESDAY	11	NOVEMBER	4	2007
2007-11-13T00:00:00Z	3	TUESDAY	11	NOVEMBER	4	2007
2007-10-23T00:00:00Z	3	TUESDAY	10	OCTOBER	4	2007
2007-08-28T00:00:00Z	3	TUESDAY	8	AUGUST	3	2007
2008-06-27T00:00:00Z	6	FRIDAY	6	JUNE	2	2008
2008-06-26T00:00:00Z	5	THURSDAY	6	JUNE	2	2008
2007-06-26T00:00:00Z	3	TUESDAY	6	JUNE	2	2007
2007-02-27T00:00:00Z	3	TUESDAY	2	FEBRUARY	1	2007
2008-02-27T00:00:00Z	4	WEDNESDAY	2	FEBRUARY	1	2008
2007-02-26T00:00:00Z	2	MONDAY	2	FEBRUARY	1	2007
2007-06-07T00:00:00Z	5	THURSDAY	6	JUNE	2	2007
2007-05-24T00:00:00Z	5	THURSDAY	5	MAY	2	2007
2008-05-12T00:00:00Z	2	MONDAY	5	MAY	2	2008
2007-12-08T00:00:00Z	7	SATURDAY	12	DECEMBER	4	2007
2007-12-06T00:00:00Z	5	THURSDAY	12	DECEMBER	4	2007
2007-03-11T00:00:00Z	1	SUNDAY	3	MARCH	1	2007
2007-06-04T00:00:00Z	2	MONDAY	6	JUNE	2	2007
2008-06-04T00:00:00Z	4	WEDNESDAY	6	JUNE	2	2008
2007-11-18T00:00:00Z	1	SUNDAY	11	NOVEMBER	4	2007
2006-02-27T00:00:00Z	2	MONDAY	2	FEBRUARY	1	2006
2007-07-21T00:00:00Z	7	SATURDAY	7	JULY	3	2007
2007-07-10T00:00:00Z	3	TUESDAY	7	JULY	3	2007
2007-07-02T00:00:00Z	2	MONDAY	7	JULY	3	2007
2007-07-01T00:00:00Z	1	SUNDAY	7	JULY	3	2007
2008-05-24T00:00:00Z	7	SATURDAY	5	MAY	2	2008
2006-03-29T00:00:00Z	4	WEDNESDAY	3	MARCH	1	2006
2008-03-29T00:00:00Z	7	SATURDAY	3	MARCH	1	2008
2007-03-29T00:00:00Z	5	THURSDAY	3	MARCH	1	2007
2007-03-20T00:00:00Z	3	TUESDAY	3	MARCH	1	2007
2004-03-20T00:00:00Z	7	SATURDAY	3	MARCH	1	2004
2007-03-13T00:00:00Z	3	TUESDAY	3	MARCH	1	2007
2007-03-12T00:00:00Z	2	MONDAY	3	MARCH	1	2007
2007-12-16T00:00:00Z	1	SUNDAY	12	DECEMBER	4	2007
2007-11-21T00:00:00Z	4	WEDNESDAY	11	NOVEMBER	4	2007
2006-11-16T00:00:00Z	5	THURSDAY	11	NOVEMBER	4	2006
2006-11-17T00:00:00Z	6	FRIDAY	11	NOVEMBER	4	2006

2007-11-10T00:00:00Z	7	SATURDAY	11	NOVEMBER	4	2007
2006-09-14T00:00:00Z	5	THURSDAY	9	SEPTEMBER	3	2006
2006-09-01T00:00:00Z	6	FRIDAY	9	SEPTEMBER	3	2006
2007-09-01T00:00:00Z	7	SATURDAY	9	SEPTEMBER	3	2007
2008-08-01T00:00:00Z	6	FRIDAY	8	AUGUST	3	2008
2006-07-27T00:00:00Z	5	THURSDAY	7	JULY	3	2006
2007-06-18T00:00:00Z	2	MONDAY	6	JUNE	2	2007
2007-06-13T00:00:00Z	4	WEDNESDAY	6	JUNE	2	2007
2007-04-11T00:00:00Z	4	WEDNESDAY	4	APRIL	2	2007
2007-12-17T00:00:00Z	2	MONDAY	12	DECEMBER	4	2007
2007-10-06T00:00:00Z	7	SATURDAY	10	OCTOBER	4	2007
2007-10-04T00:00:00Z	5	THURSDAY	10	OCTOBER	4	2007
2006-11-07T00:00:00Z	3	TUESDAY	11	NOVEMBER	4	2006
2007-10-31T00:00:00Z	4	WEDNESDAY	10	OCTOBER	4	2007

III. Create CUSTOMER_COPY table

Make a copy of the OE.CUSTOMER table (Create a table for this - example,CUSTOMER_COPY)

For analysis purposes, Decision-Makers want to classify the Customers according to the Turnover of each one according to the following conditions:

- Turnover of a customer is the total amount of all his orders made. This data is not in the database table, therefore, you need to calculate it in order to determine their customer type
- The type of the customer depends on his Turnover. We expect 3 types of customers:
 - Type 'A' - if the Turnover $\geq$ Avg_Turnover (Avg_Turnover is the average turnover of all customers)
 - Type 'B' - if $\frac{1}{2} * \text{Avg_Turnover} \leq \text{Turnover} < \text{Avg_Turnover}$; and
 - Type 'C' - any other value of Turnover

ETL Process - Task 1: Calculate the Turnover and the Type of the customer

1. Make a copy of the OE.CUSTOMER table

```
CREATE TABLE CUSTOMER_COPY AS
SELECT CUSTOMER_ID, CUST_FIRST_NAME, CUST_LAST_NAME, NLS_LANGUAGE,
NLS_TERRITORY, CREDIT_LIMIT, CUST_EMAIL, ACCOUNT_MGR_ID,
CUST_GEO_LOCATION, DATE_OF_BIRTH, MARITAL_STATUS, GENDER, INCOME_LEVEL
FROM OE.CUSTOMERS;
```

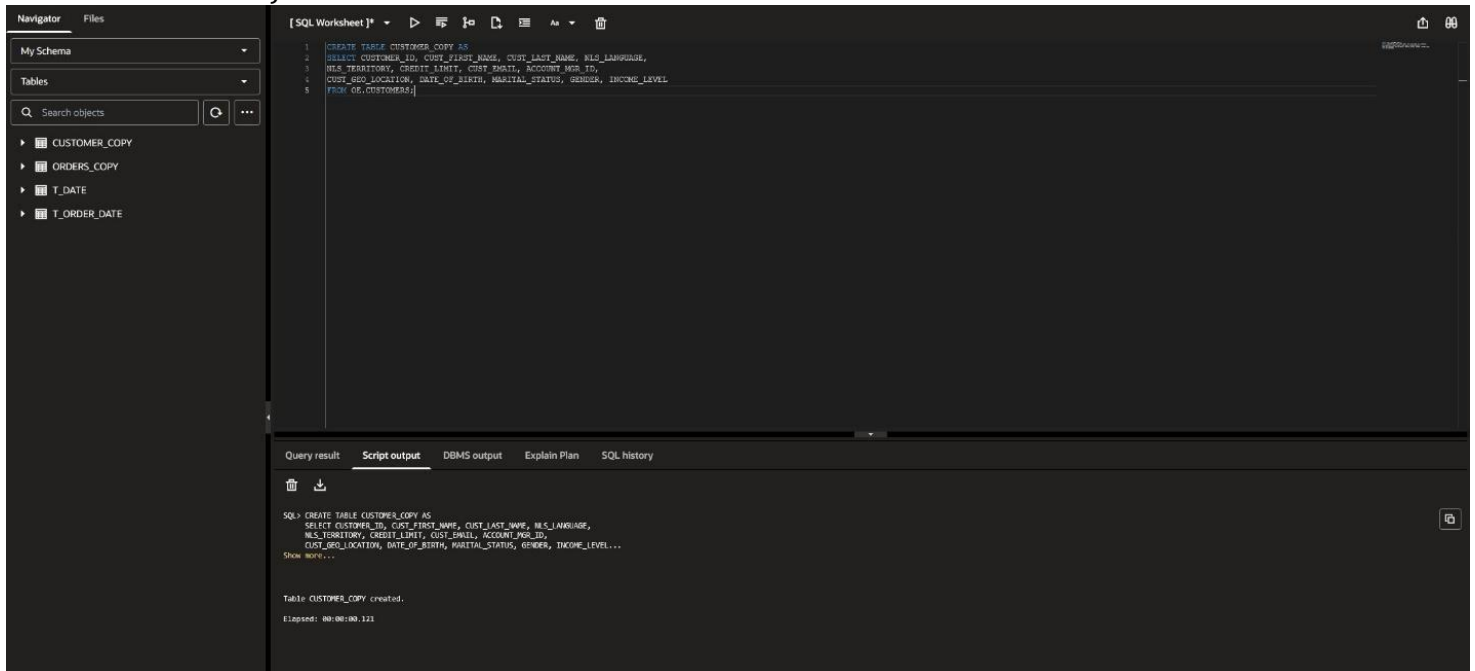


Table CUSTOMER_COPY created.

2. Addition of TURNOVER (12,2) Column

Add a column TURNOVER NUMBER (12,2) to your new table. To do this you need to modify the structure of your table (i.e. you need to add a new column) Note - TURNOVER is a NUMBER type of data, allowing 12 digit inclusive of 2 decimal places

ALTER TABLE CUSTOMER_COPY **ADD** TURNOVER **NUMBER**(12,2);

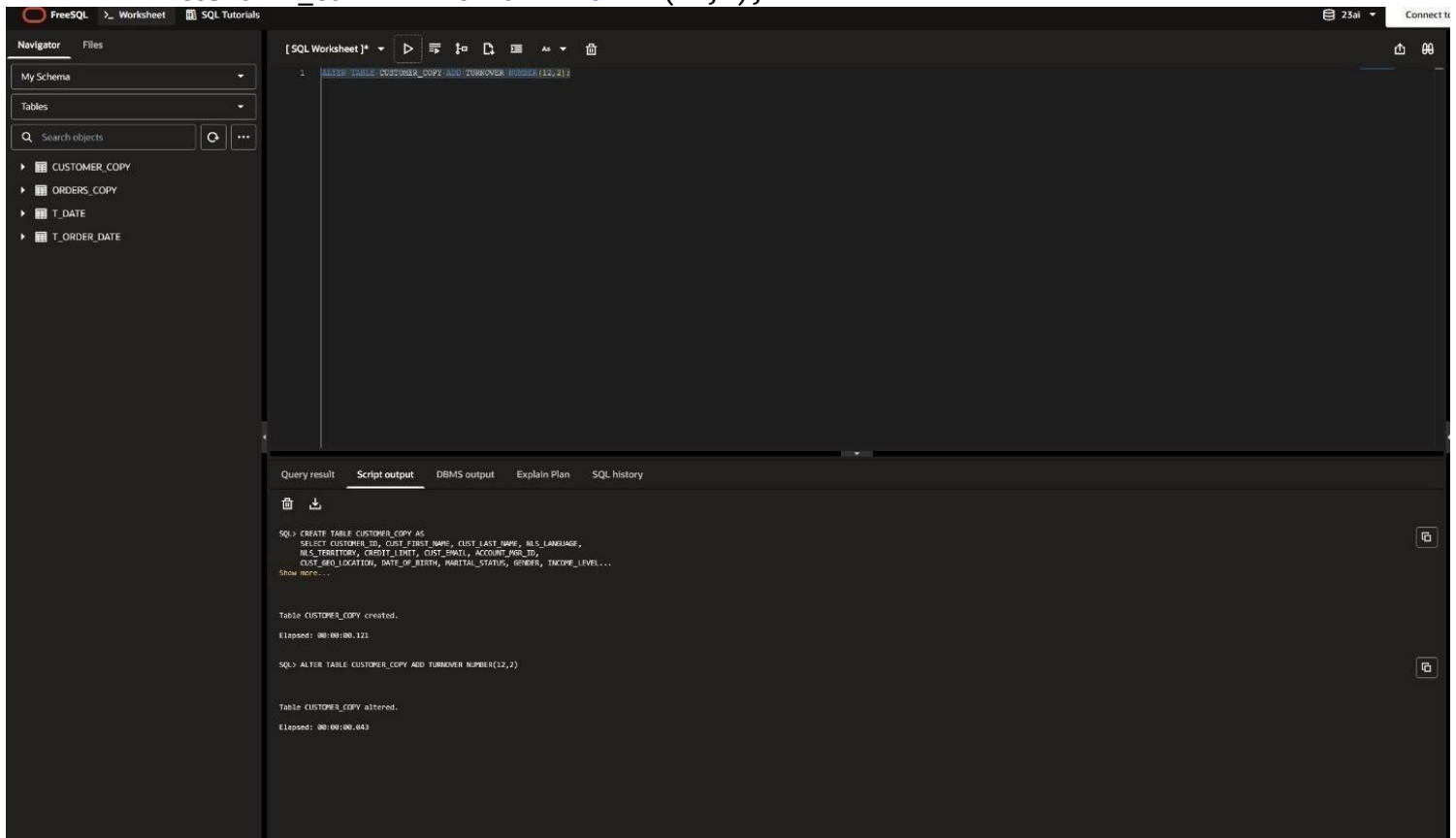


Table CUSTOMER_COPY altered.

3. Addition of CUST_TYPE (VARCHAR) Column

Add another column called CUST_TYPE with type VARCHAR(1) to your table. This column will store the type of the customer ('A', 'B', or 'C') as mentioned above.

```
ALTER TABLE CUSTOMER_COPY ADD CUST_TYPE VARCHAR2(1);
```

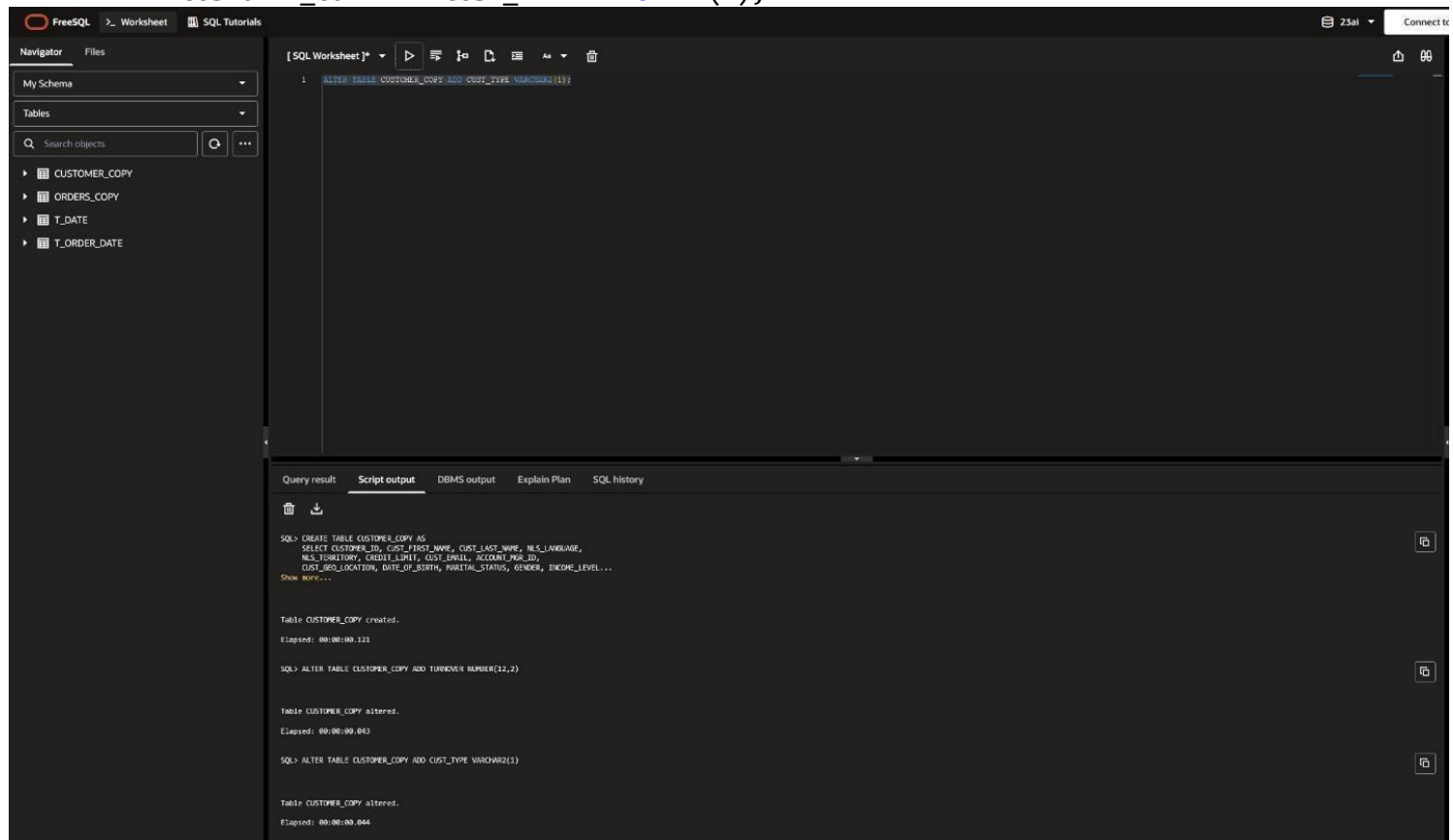


Table CUSTOMER_COPY altered.

4. Correlated Update Statement for Turnover Calculation

Write SQL correlated statement to populate the TURNOVER column. To do this, you must understand the data set. Run the SQL statement and analyze the data in the table to understand the data.

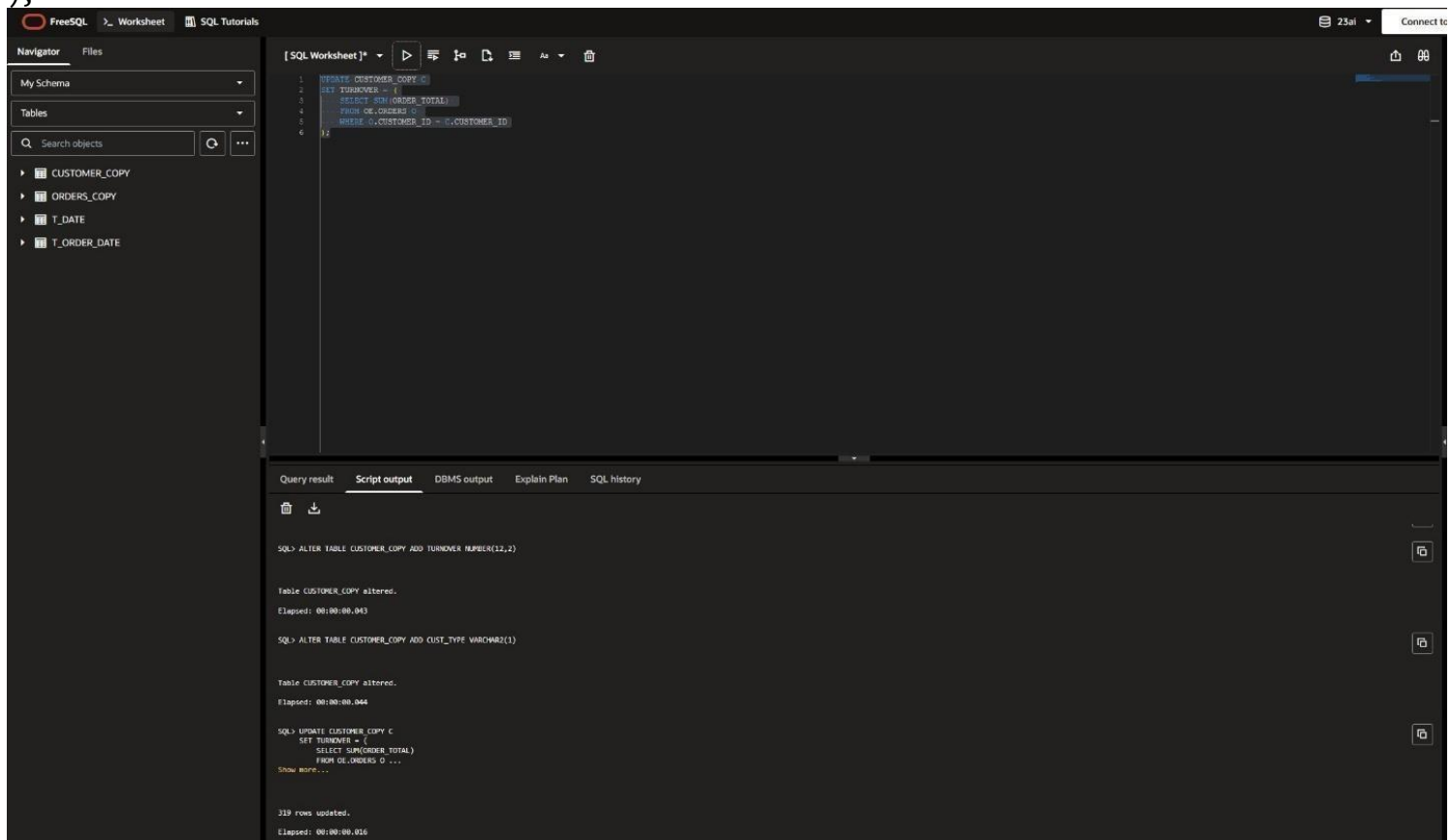
The table CUSTOMER_COPY contains unique data about the customer - therefore you need to sum-up the amount of sales made by the customer, and place the value in the column TURNOVER.

The data of the sales order for every customer is located in a table called OE.ORDERS (original data source).

Check the table OE.ORDERS and you will see that every customer made more than one order (at a different date and time of course), and that each of these orders has its own ORDER_ID.

Write the statement to update (in this case) CUSTOMER_COPY table with the populated value of orders by each customer.

```
UPDATE CUSTOMER_COPY C
SET TURNOVER = (
    SELECT SUM(ORDER_TOTAL)
    FROM OE.ORDERS O
    WHERE O.CUSTOMER_ID = C.CUSTOMER_ID
);
```

A screenshot of a SQL IDE interface. The top-left pane shows a 'Navigator' with a tree view of a schema containing tables: CUSTOMER_COPY, ORDERS_COPY, T_DATE, and T_ORDER_DATE. The main editor pane shows a SQL script with the following lines:

```
1 UPDATE CUSTOMER_COPY C
2 SET TURNOVER = (
3     SELECT SUM(ORDER_TOTAL)
4     FROM OE.ORDERS O
5     WHERE O.CUSTOMER_ID = C.CUSTOMER_ID
6 );
```

The bottom pane shows the 'Script output' tab with the following text:

```
SQL> ALTER TABLE CUSTOMER_COPY ADD TURNOVER NUMBER(12,2)

Table CUSTOMER_COPY altered.
Elapsed: 00:00:00.043

SQL> ALTER TABLE CUSTOMER_COPY ADD CUST_TYPE VARCHAR2(1)

Table CUSTOMER_COPY altered.
Elapsed: 00:00:00.044

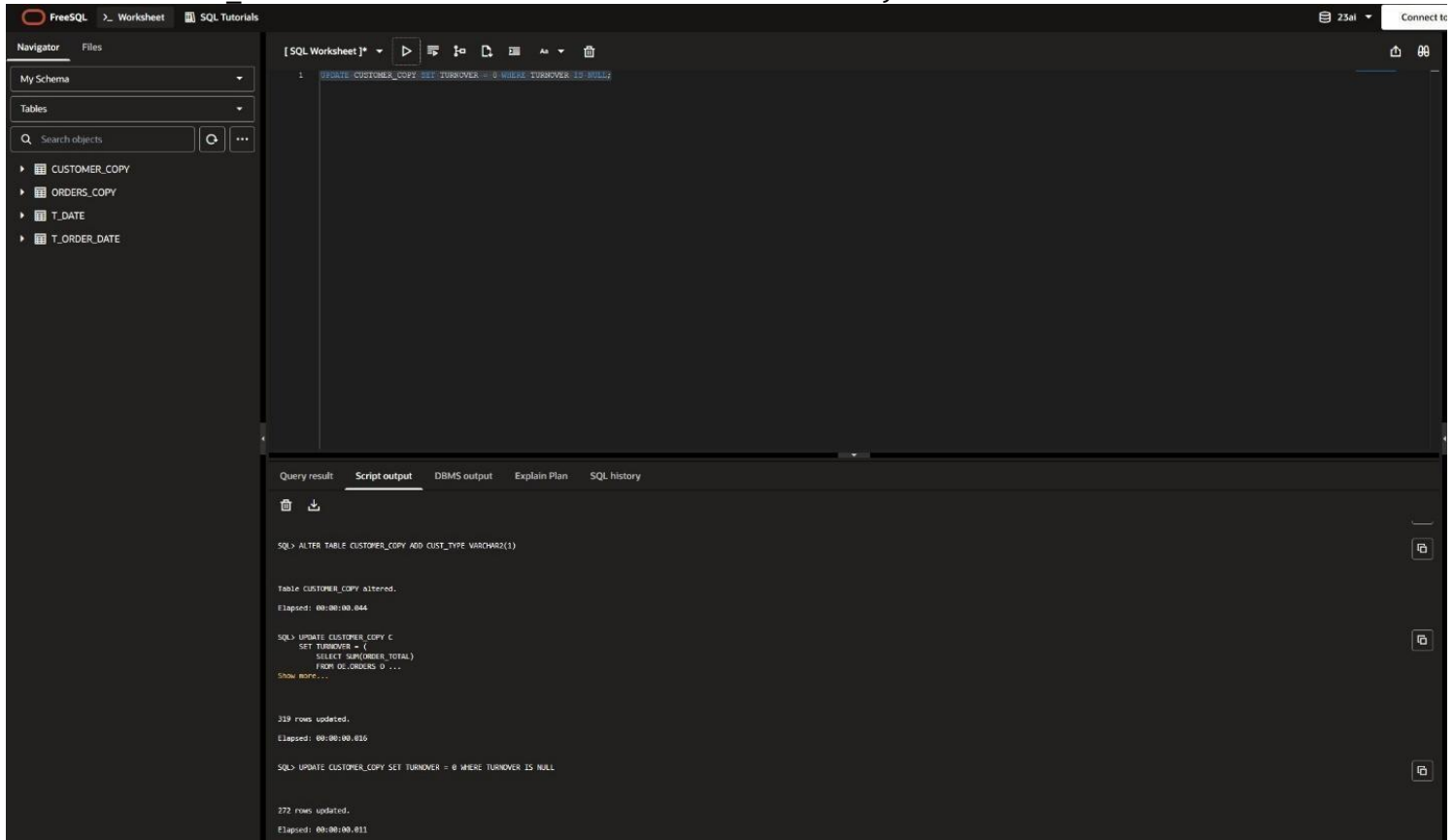
SQL> UPDATE CUSTOMER_COPY C
SET TURNOVER = (
    SELECT SUM(ORDER_TOTAL)
    FROM OE.ORDERS O ...
Show more...

319 rows updated.
Elapsed: 00:00:00.016
```

319 rows updated.

To handle customers with 0 turnover

```
UPDATE CUSTOMER_COPY SET TURNOVER = 0 WHERE TURNOVER IS NULL;
```



The screenshot shows the FreeSQL SQL Worksheet interface. The left sidebar contains a 'Navigator' pane with 'My Schema' and 'Tables' sections. Under 'Tables', there is a search bar and a list of tables: CUSTOMER_COPY, ORDERS_COPY, T_DATE, and T_ORDER_DATE. The main workspace displays a SQL query: `UPDATE CUSTOMER_COPY SET TURNOVER = 0 WHERE TURNOVER IS NULL;`. Below the query, the 'Script output' pane shows the execution results. The output includes the following text:
SQL> ALTER TABLE CUSTOMER_COPY ADD CUST_TYPE VARCHAR2(1)

Table CUSTOMER_COPY altered.
Elapsed: 00:00:00.044

SQL> UPDATE CUSTOMER_COPY C
SET TURNOVER = (
SELECT SUM(ORDER_TOTAL)
FROM OE.ORDERS O ...
Show more...

319 rows updated.
Elapsed: 00:00:00.016

SQL> UPDATE CUSTOMER_COPY SET TURNOVER = 0 WHERE TURNOVER IS NULL

272 rows updated.
Elapsed: 00:00:00.011

272 rows updated.

5. PL/SQL Block Implementation for Classification (Types A, B, and C)

Use PL/SQL block command to set the customer type according to the rules explained above. You need to use IF statement to check which category they fall into, and UPDATE the CUSTOMER_COPY table with the correct type of customer.

DECLARE

v_avg_turnover NUMBER;

BEGIN

-- Calculate Avg

SELECT AVG(TURNOVER) INTO v_avg_turnover FROM CUSTOMER_COPY;

FOR record IN (SELECT * FROM CUSTOMER_COPY) LOOP

IF record.TURNOVER >= v_avg_turnover THEN

UPDATE CUSTOMER_COPY SET CUST_TYPE = 'A' WHERE CUSTOMER_ID = record.CUSTOMER_ID;

ELSIF record.TURNOVER >= (0.5 * v_avg_turnover) THEN

UPDATE CUSTOMER_COPY SET CUST_TYPE = 'B' WHERE CUSTOMER_ID = record.CUSTOMER_ID;

ELSE

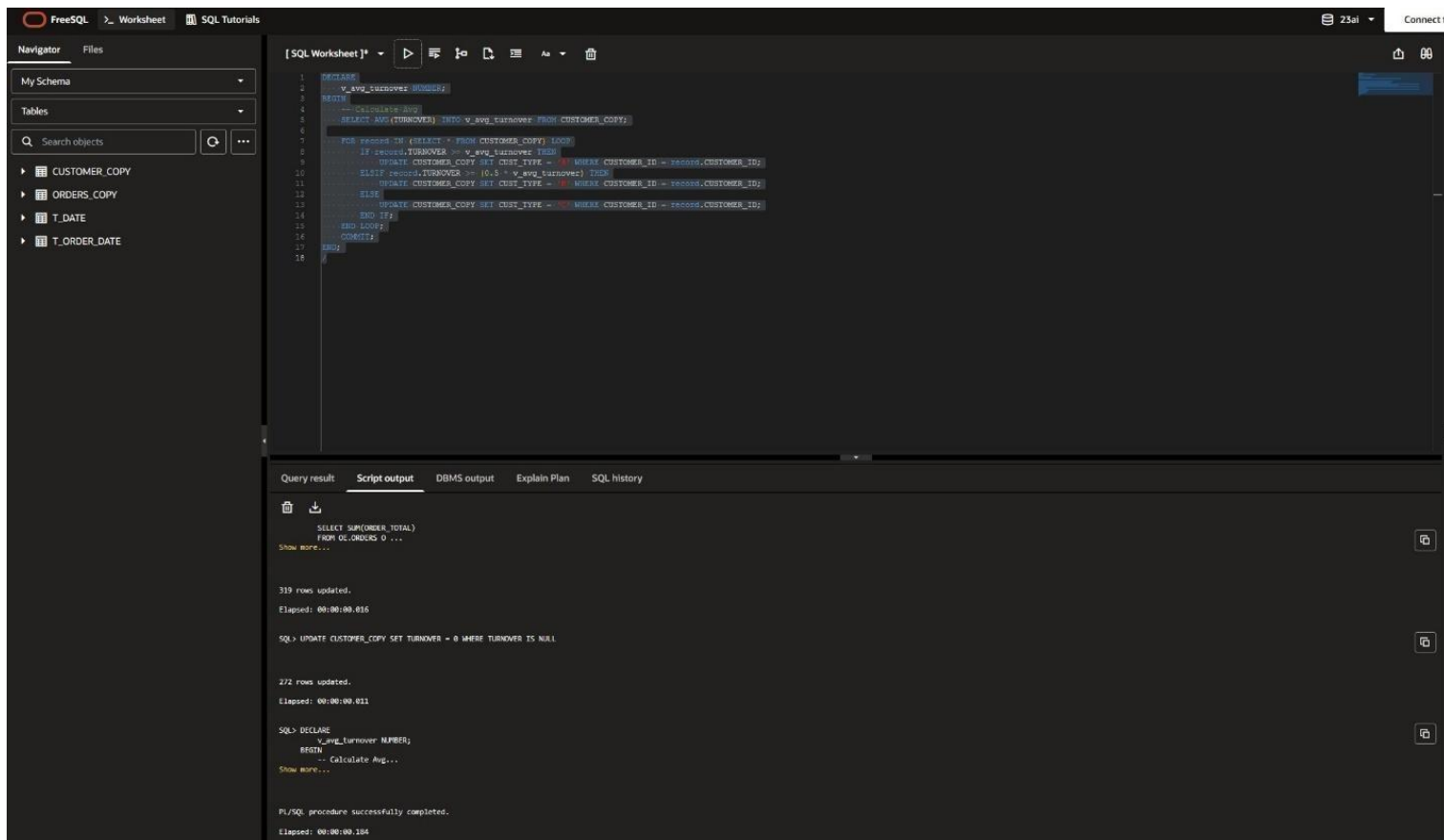
UPDATE CUSTOMER_COPY SET CUST_TYPE = 'C' WHERE CUSTOMER_ID = record.CUSTOMER_ID;

END IF;

END LOOP;

COMMIT;

END;



```
1 DECLARE
2   v_avg_turnover NUMBER;
3 BEGIN
4   -- Calculate Avg
5   SELECT AVG(TURNOVER) INTO v_avg_turnover FROM CUSTOMER_COPY;
6
7   FOR record IN (SELECT * FROM CUSTOMER_COPY) LOOP
8     IF record.TURNOVER >= v_avg_turnover THEN
9       UPDATE CUSTOMER_COPY SET CUST_TYPE = 'A' WHERE CUSTOMER_ID = record.CUSTOMER_ID;
10    ELSIF record.TURNOVER >= (0.5 * v_avg_turnover) THEN
11      UPDATE CUSTOMER_COPY SET CUST_TYPE = 'B' WHERE CUSTOMER_ID = record.CUSTOMER_ID;
12    ELSE
13      UPDATE CUSTOMER_COPY SET CUST_TYPE = 'C' WHERE CUSTOMER_ID = record.CUSTOMER_ID;
14    END IF;
15  END LOOP;
16  COMMIT;
17 END;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

SELECT SUM(ORDER_TOTAL)
FROM CC.ORDERS O ...
Show more...

319 rows updated.
Elapsed: 00:00:00.016

SQL> UPDATE CUSTOMER_COPY SET TURNOVER = 0 WHERE TURNOVER IS NULL

272 rows updated.
Elapsed: 00:00:00.011

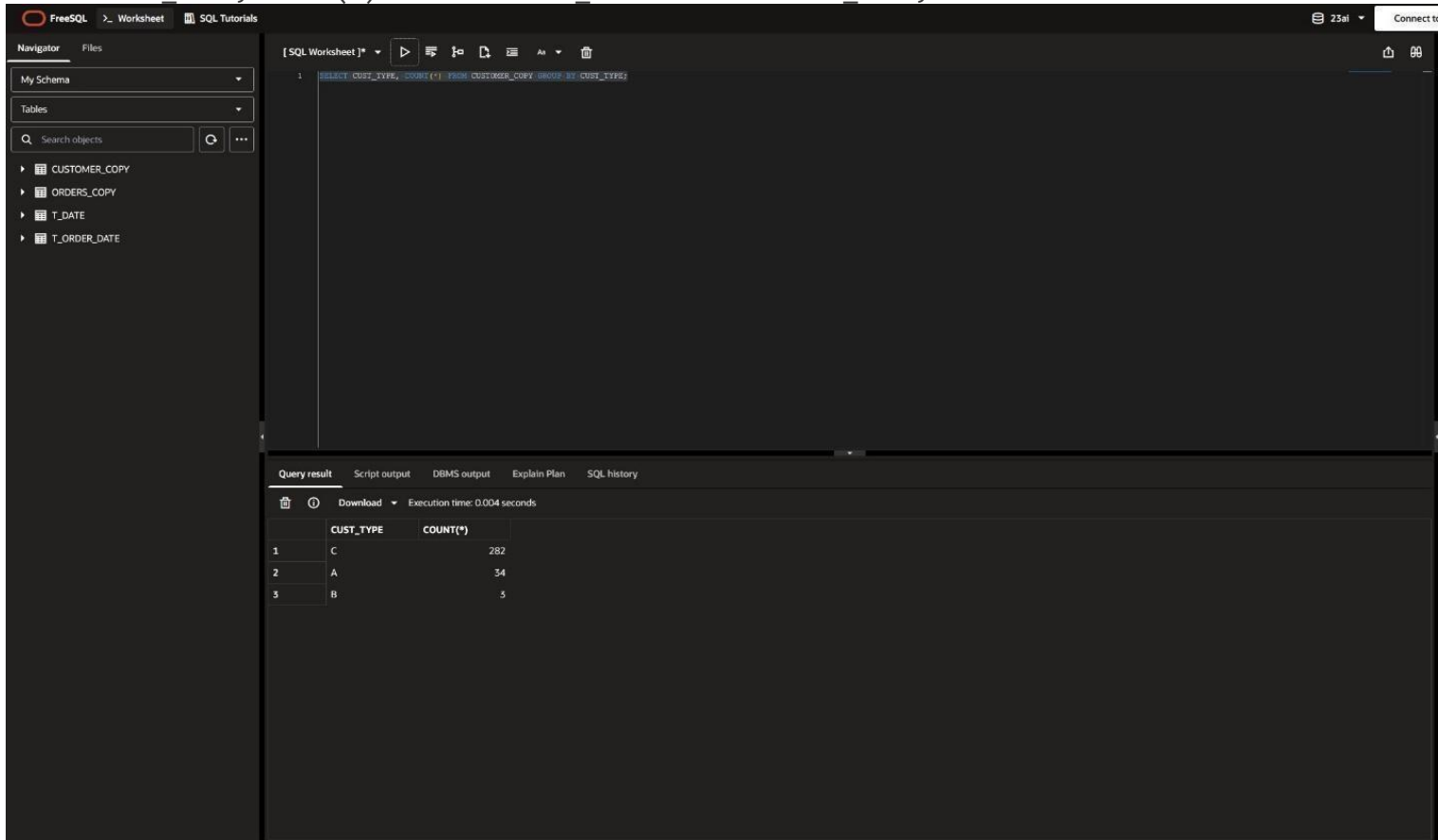
SQL> DECLARE
v_avg_turnover NUMBER;
BEGIN
-- Calculate Avg...
Show more...

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.186

PL/SQL procedure successfully completed.

Display the number of customers in every group A, B and C.

```
SELECT CUST_TYPE, COUNT(*) FROM CUSTOMER_COPY GROUP BY CUST_TYPE;
```



The screenshot shows the FreeSQL SQL Worksheet interface. On the left is a Navigator pane with 'My Schema' and 'Tables' sections. The 'Tables' section lists 'CUSTOMER_COPY', 'ORDERS_COPY', 'T_DATE', and 'T_ORDER_DATE'. The main editor area contains the SQL query: `SELECT CUST_TYPE, COUNT(*) FROM CUSTOMER_COPY GROUP BY CUST_TYPE;`. Below the editor is a 'Query result' pane showing the execution time as 0.004 seconds and a table with 3 rows and 2 columns: 'CUST_TYPE' and 'COUNT(*)'.

	CUST_TYPE	COUNT(*)
1	C	282
2	A	34
3	B	3

```
CUST_TYPE COUNT(*)
```

```
-----
```

```
C          282
```

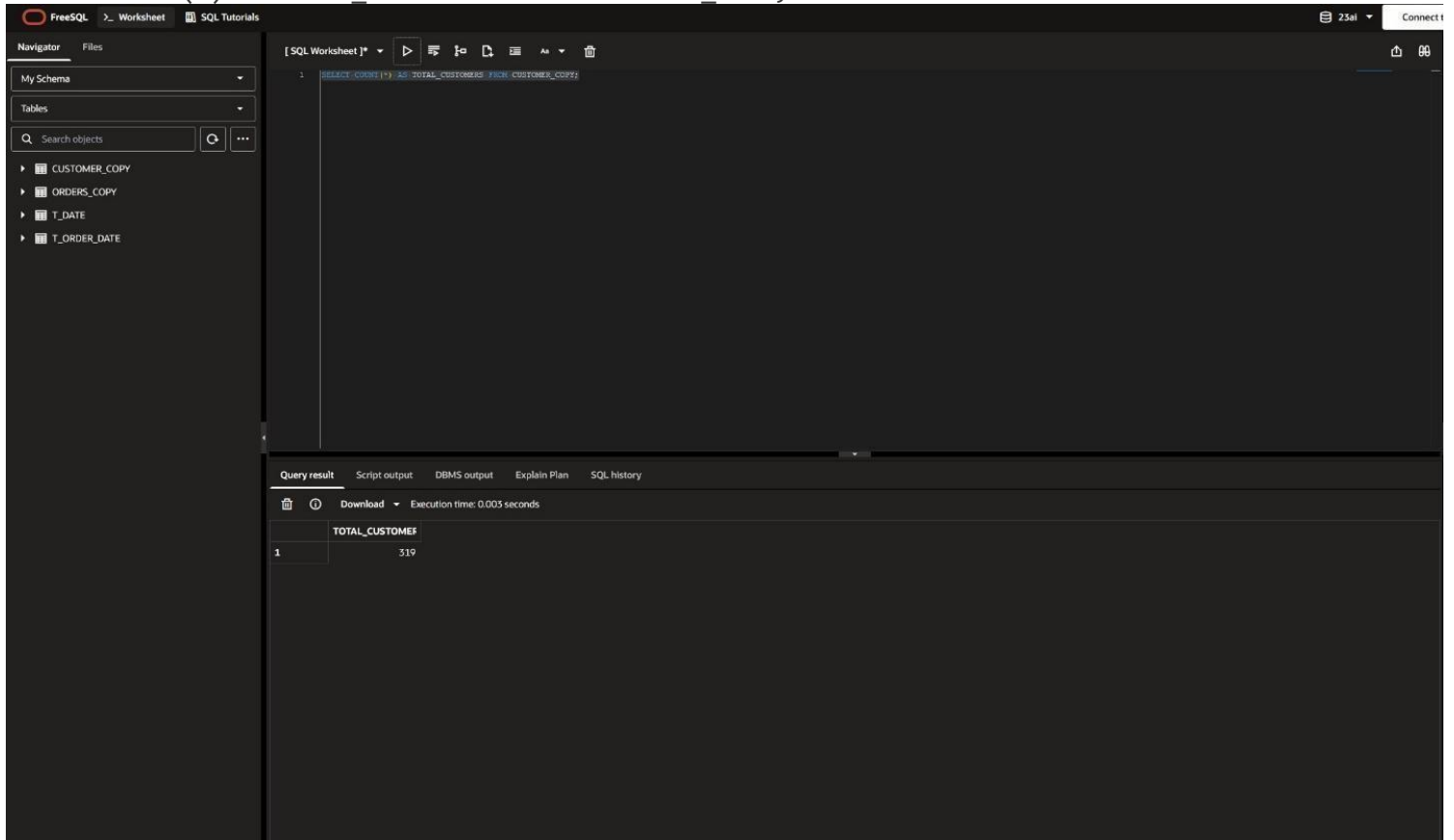
```
A          34
```

```
B           3
```

```
3 rows selected.
```

Display the total number of customers.

```
SELECT COUNT(*) AS TOTAL_CUSTOMERS FROM CUSTOMER_COPY;
```



The screenshot shows the FreeSQL SQL client interface. On the left is a Navigator pane with a tree view showing a schema named 'My Schema' containing four tables: 'CUSTOMER_COPY', 'ORDERS_COPY', 'T_DATE', and 'T_ORDER_DATE'. The main editor area displays the SQL query: `SELECT COUNT(*) AS TOTAL_CUSTOMERS FROM CUSTOMER_COPY;`. Below the editor is a 'Query result' pane showing the execution output. It indicates 'Execution time: 0.003 seconds' and displays a single row with the column 'TOTAL_CUSTOMERS' and the value '319'.

	TOTAL_CUSTOMERS
1	319

TOTAL_CUSTOMERS

319

1 rows selected.

ETL Process - Task 2: Currency Conversion s ORDERS_IN_EURO

During the ETL process, you may need to convert source data values from one unit of measure to another; this occurs mainly when you load your DW from heterogeneous data sources.

Standardizing an Amount of Money by Conversion to Standard Currency

Suppose the column OE.ORDERS.ORDER_TOTAL is the amount of the order expressed in the currency of the country (OE.CUSTOMER.NLS_TERRITORY) of the Customer.

This means, when the OE.CUSTOMER.NLS_TERRITORY is:

- CHINA the amount is in CHY,
- INDIA the amount is in INR,
- GERMANY and ITALY the amount is in EUR (Euro), etc.

In fact, there are eight (8) different Countries in the OE.CUSTOMERS.NLS_TERRITORY column.

1. Identification of Distinct Customer Territories

Write your code to show how many countries the customers are coming from?

```
SELECT DISTINCT NLS_TERRITORY FROM OE.CUSTOMERS;
```

The screenshot shows the Oracle LiveSQL interface. At the top, the title bar says "Oracle LiveSQL". Below it, there are buttons for "Run", "Format", "Clear", and "History". The "Run" button is highlighted. Below the buttons, there are tabs for "SQL" and "Results". The "Results" tab is selected, showing the output of the query `SELECT DISTINCT NLS_TERRITORY FROM OE.CUSTOMERS;`. The output is a table with one column, **NLS_TERRITORY**, and 12 rows. The rows contain the following values: CHINA, INDIA, SWITZERLAND, THAILAND, JAPAN, ITALY, GERMANY, UNITED STATES OF AMERICA, FRANCE, UNITED KINGDOM, AUSTRALIA, and BRAZIL. Below the table, it says "12 rows selected".

NLS_TERRITORY
CHINA
INDIA
SWITZERLAND
THAILAND
JAPAN
ITALY
GERMANY
UNITED STATES OF AMERICA
FRANCE
UNITED KINGDOM
AUSTRALIA
BRAZIL

NLS_TERRITORY
AMERICA
CHINA
INDIA
GERMANY
ITALY
SWITZERLAND
THAILAND

JAPAN

8 rows selected.

Before we sum these total (like u already completed in TASK 1) in actual practice, you need to convert these totals to the same currency.

Let us convert these totals from their own currency (based on NLS_TERRITORY, which is the country) to Euro.

To do this we need to prepare the following table for exchange rate of these currencies to Euro.

CUR_SIGN	COUNTRY	CUR_QTY	EURO_RATE
CNY	CHINA	1	0.13
INR	INDA	1	0.011
CHF	SWITZERLAN	1	0.92
THB	THAILAND	1	0.027
JPY	JAPAN	1	0.0079
EUR	ITALY	1	1
USD	AMERICA	1	0.81
EUR	GERMANY	1	1

Execute the code(script) below to create the **CURRENCY_RATES** table and insert the 8 lines above. Show the data from your table **CURRENCY_RATES**.

```
CREATE TABLE CURRENCY_RATES(  
    CUR_SIGN CHAR(3),  
    COUNTRY VARCHAR2(30),  
    CUR_QTY NUMBER(4),  
    EURO_RATE NUMBER(8,6)  
);
```

The screenshot shows the Oracle LiveSQL web interface. At the top, there's a header with the Oracle logo and 'LiveSQL' text. Below the header, there are buttons for 'Run', 'Format', 'Clear', and 'History'. The 'Run' button is highlighted. Below these buttons, there's a tabbed interface with 'SQL' and 'Results' tabs. The 'SQL' tab is active, showing the SQL script that was executed: `CREATE TABLE CURRENCY_RATES (
CUR_SIGN CHAR(3),
COUNTRY VARCHAR2(30),
CUR_QTY NUMBER(4),
EURO_RATE NUMBER(8,6)
);`. Below the SQL tab, there's a green status bar with a checkmark icon and the text 'Table CURRENCY_RATES created.'

Table CURRENCY_RATES created.

```

INSERT INTO CURRENCY_RATES VALUES ('CNY', 'CHINA', 1, 0.13);
INSERT INTO CURRENCY_RATES VALUES ('INR', 'INDIA', 1, 0.011);
INSERT INTO CURRENCY_RATES VALUES ('CHF', 'SWITZERLAND', 1, 0.92);
INSERT INTO CURRENCY_RATES VALUES ('THB', 'THAILAND', 1, 0.027);
INSERT INTO CURRENCY_RATES VALUES ('JPY', 'JAPAN', 1, 0.0079);
INSERT INTO CURRENCY_RATES VALUES ('EUR', 'ITALY', 1, 1);
INSERT INTO CURRENCY_RATES VALUES ('USD', 'AMERICA', 1, 0.81);
INSERT INTO CURRENCY_RATES VALUES ('EUR', 'GERMANY', 1, 1);

```

Oracle LiveSQL

Run

Format

Clear

History

SQL	Results
INSERT INTO CURRENCY_RATES VALUES ('CNY', 'CHINA', 1, 0.13);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('INR', 'INDIA', 1, 0.011);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('CHF', 'SWITZERLAND', 1, 0.92);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('THB', 'THAILAND', 1, 0.027);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('JPY', 'JAPAN', 1, 0.0079);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('EUR', 'ITALY', 1, 1);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('USD', 'AMERICA', 1, 0.81);	✓ 1 row inserted.
INSERT INTO CURRENCY_RATES VALUES ('EUR', 'GERMANY', 1, 1);	✓ 1 row inserted.

```

COMMIT;

1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.

```

2. Development of ORDERS_IN_EURO Staging Table

Use the CREATE and SELECT statements to create, from the tables ORDERS and CUSTOMERS, anew table called ORDERS_IN_EURO (ORDER_ID, CUSTOMER_ID, COUNTRY, ORDER_TOTAL, ORDER_TOTAL_EURO).

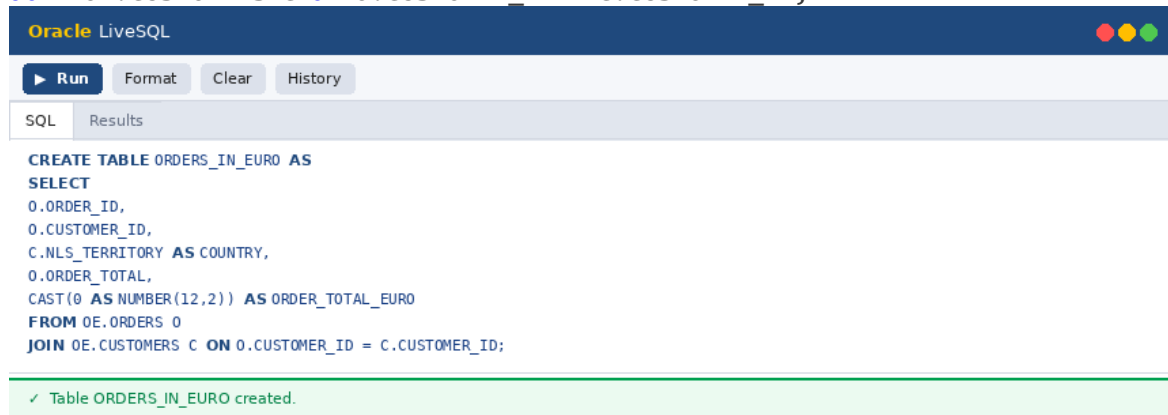
```
CREATE TABLE ORDERS_IN_EURO AS
```

```
SELECT
```

```
    O.ORDER_ID,  
    O.CUSTOMER_ID,  
    C.NLS_TERRITORY AS COUNTRY,  
    O.ORDER_TOTAL,  
    CAST(O AS NUMBER(12,2)) AS ORDER_TOTAL_EURO
```

```
FROM OE.ORDERS O
```

```
JOIN OE.CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID;
```



The screenshot shows the Oracle LiveSQL interface. At the top, there's a dark blue header with the 'Oracle LiveSQL' logo and three window control buttons (red, yellow, green). Below the header is a toolbar with buttons for 'Run' (with a play icon), 'Format', 'Clear', and 'History'. The main area is divided into two tabs: 'SQL' and 'Results'. The 'SQL' tab is active, displaying the SQL script that was entered. Below the script, a green status bar indicates '✓ Table ORDERS_IN_EURO created.'

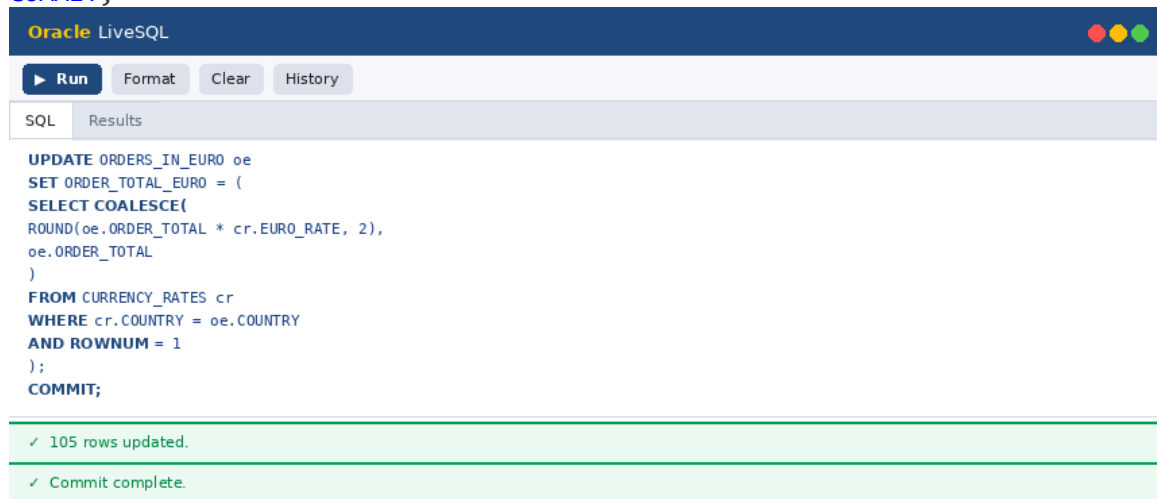
```
Oracle LiveSQL  
Run Format Clear History  
SQL Results  
CREATE TABLE ORDERS_IN_EURO AS  
SELECT  
    O.ORDER_ID,  
    O.CUSTOMER_ID,  
    C.NLS_TERRITORY AS COUNTRY,  
    O.ORDER_TOTAL,  
    CAST(O AS NUMBER(12,2)) AS ORDER_TOTAL_EURO  
FROM OE.ORDERS O  
JOIN OE.CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID;  
✓ Table ORDERS_IN_EURO created.
```

Table ORDERS_IN_EURO created.

3. PL/SQL Logic for Multi-Currency Euro Conversion

Write a PL/SQL function/code to populate the column ORDER_TOTAL_EURO that will contain the total of the order converted to EURO.

```
UPDATE ORDERS_IN_EURO oe
SET ORDER_TOTAL_EURO = (
  SELECT COALESCE(
    ROUND(oe.ORDER_TOTAL * cr.EURO_RATE, 2),
    oe.ORDER_TOTAL)
  FROM CURRENCY_RATES cr
  WHERE cr.COUNTRY = oe.COUNTRY AND ROWNUM = 1);
COMMIT;
```



The screenshot shows the Oracle LiveSQL interface. At the top, there's a dark blue header with the 'Oracle LiveSQL' logo and three colored window control buttons (red, yellow, green). Below the header is a toolbar with buttons for 'Run', 'Format', 'Clear', and 'History'. The main area is divided into two tabs: 'SQL' and 'Results'. The 'SQL' tab is active, displaying the following SQL code:

```
UPDATE ORDERS_IN_EURO oe
SET ORDER_TOTAL_EURO = (
  SELECT COALESCE(
    ROUND(oe.ORDER_TOTAL * cr.EURO_RATE, 2),
    oe.ORDER_TOTAL
  )
  FROM CURRENCY_RATES cr
  WHERE cr.COUNTRY = oe.COUNTRY
  AND ROWNUM = 1
);
COMMIT;
```

Below the SQL tab, the 'Results' tab is active, showing two green status messages:

- ✓ 105 rows updated.
- ✓ Commit complete.

105 rows updated.

Display the content of your table (or some data rows) to check that the content in the ORDER_TOTAL_EURO is correct.

```
SELECT * FROM ORDERS_IN_EURO WHERE ROWNUM <= 10;
```

Oracle LiveSQL

Run Format Clear History

SQL Results

SELECT * FROM ORDERS_IN_EURO WHERE ROWNUM <= 10;

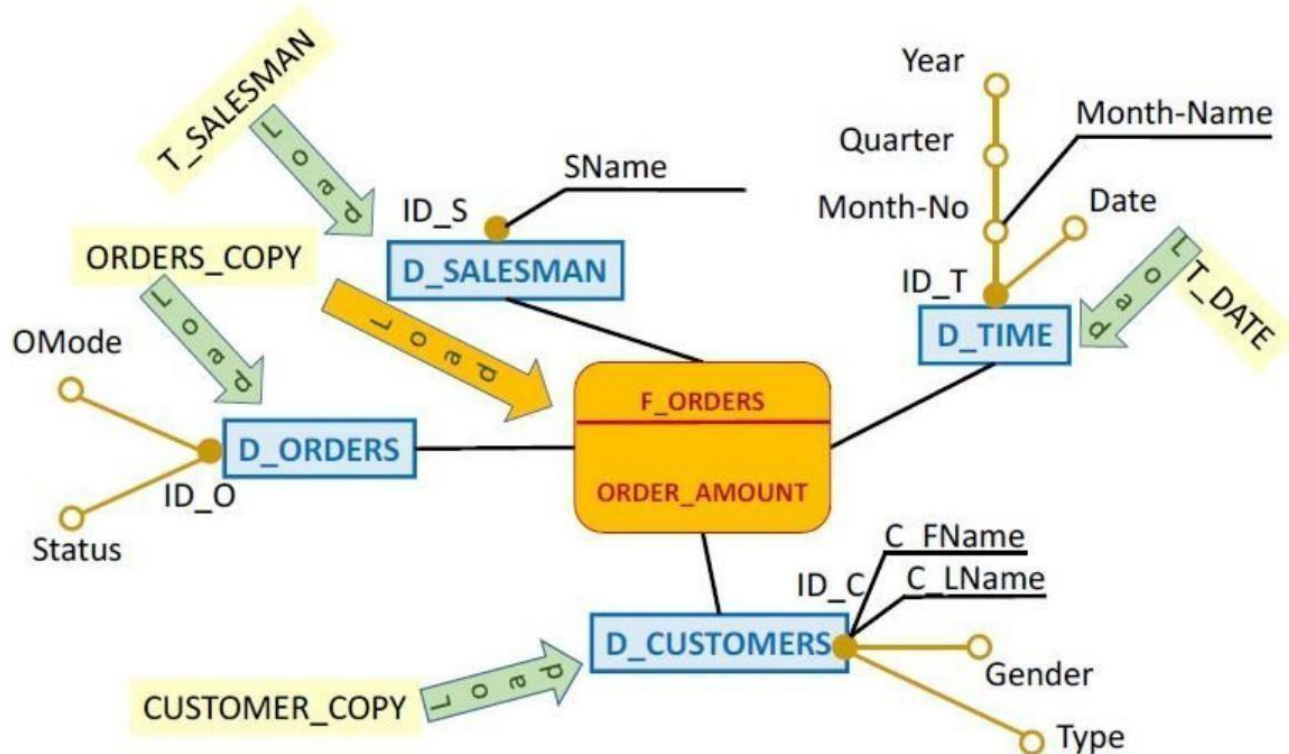
ORDER_ID	CUST_ID	COUNTRY	ORDER_TOTAL	ORDER_TOTAL_EURO
2354	101	GERMANY	35987.60	35987.60
2355	102	UNITED STATES OF AMERICA	18560.00	15033.60
2356	103	JAPAN	4200.00	33.18
2357	104	ITALY	12800.00	12800.00
2358	105	CHINA	9650.00	1254.50
2359	106	INDIA	7320.00	80.52
2360	107	SWITZERLAND	22100.00	20332.00
2361	108	THAILAND	5400.00	145.80
2362	109	GERMANY	8750.00	8750.00
2363	110	JAPAN	3100.00	24.49

10 rows selected

ORDER_ID	CUSTOMER_ID	COUNTRY	ORDER_TOTAL	ORDER_TOTAL_EURO
2458	101	AMERICA	78279.6	63406.48
2447	101	AMERICA	33893.6	27453.82
2413	101	AMERICA	48552	39327.12
2430	101	AMERICA	29669.9	24032.62
2397	102	AMERICA	42283.2	34249.39
2432	102	AMERICA	10523	8523.63
2414	102	AMERICA	10794.6	8743.63
2431	102	AMERICA	5610.6	4544.59
2454	103	AMERICA	6653.4	5389.25
2433	103	AMERICA	78	63.18

10 rows selected.

Task 3: Star Schema Implementation



We want to load a Star Schema as shown above. To do this you must use the tables that you have prepared in previous project - T_DATE, CUSTOMERS_COPY, and ORDERS_COPY.

1. Create T_SALESMAN table

Because the OE database does not have a SALESMAN table, we need to CREATE one called T_SALESMAN (SALES_REP_ID, SNAME) from the OE.ORDERS.SALES_REP_ID; its columns are:

- SALES_REP_ID unique values. Replace Null IDs with 999.
- SNAME is a new column. Set the SNAME as the concatenation of the SALES_REP_ID and the constant 'NAME' (for example, Salesman 153 will have name 'NAME-153').

```
CREATE TABLE T_SALESMAN AS
```

```
SELECT DISTINCT
```

```
  NVL(SALES_REP_ID, 999) AS SALES_REP_ID,  
  'NAME-' || NVL(SALES_REP_ID, 999) AS SNAME
```

```
FROM OE.ORDERS
```

The screenshot shows the FreeSQL SQL Worksheet interface. The SQL script is entered in the editor and executed. The 'Script output' tab is active, displaying the following results:

```
SQL> CREATE TABLE T_SALESMAN AS  
      SELECT DISTINCT  
        NVL(SALES_REP_ID, 999) AS SALES_REP_ID,  
        'NAME-' || NVL(SALES_REP_ID, 999) AS SNAME...
```

Below the script output, the message "Table T_SALESMAN created." is displayed, along with the execution time "Elapsed: 00:00:00.217".

The left sidebar shows the database schema with the following tables listed:

- MY_PRODUCT_INFO
- ORDERS_COPY
- ORDERS_DIRECT
- ORDERS_EURO
- ORDERS_IN_EURO
- ORDERS_ONLINE
- T_DATE
- T_ORDER_DATE
- T_SALESMAN**

The right sidebar shows a search bar and a list of tutorials, including "Creating..." with 4 likes and 11.3K views.

Table T_SALESMAN created

```
SELECT * FROM T_SALESMAN;
```

SALES_REP_ID	SNAME
153	NAME-153
154	NAME-154
155	NAME-155
156	NAME-156
158	NAME-158
159	NAME-159
160	NAME-160
161	NAME-161
163	NAME-163
999	NAME-999

10 rows selected.

2. Create Dimensional Tables

Use the CREATE with an embedded SELECT statement (and tables T_DATE, CUSTOMERS_COPY, ORDERS_COPY, and T_SALESMAN) to create and load the four-dimensional tables D_ORDERS, D_CUSTOMERS, D_TIME and D_SALESMAN as they are described in the figure shows above.

For each Dimension, define the PK constraint and name it as follows:

- PK_D_ORDERS Primary Key for D_ORDERS
- PK_D_SALESMAN Primary Key for D_SALESMAN
- PK_D_TIME Primary Key for D_TIME
- PK_D_CUSTOMERS Primary Key for D_CUSTOMERS

a) Handle D_ORDERS Table

1) Create Table

```
CREATE TABLE D_ORDERS AS
  SELECT ORDER_ID AS ID_O, ORDER_MODE AS OMode, ORDER_STATUS AS Status
FROM ORDERS_COPY;
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a list of tables in the 'My Schema' database, with 'D_ORDERS' highlighted. The main workspace shows a SQL worksheet with the following query:

```
1 CREATE TABLE D_ORDERS AS
2 | SELECT ORDER_ID AS ID_O, ORDER_MODE AS OMode, ORDER_STATUS AS Status
3 | FROM ORDERS_COPY;
```

Below the query, the 'Script output' tab is active, showing the execution results:

```
Table T_SALESMAN created.
Elapsed: 00:00:00.217

SQL> CREATE TABLE D_ORDERS AS
      SELECT ORDER_ID AS ID_O, ORDER_MODE AS OMode, ORDER_STATUS AS Status
      FROM ORDERS_COPY

Table D_ORDERS created.
Elapsed: 00:00:00.180
```

On the right side of the interface, there is a 'SQL Tutorials' sidebar with a search bar and a list of tutorials. The first tutorial, 'Creating...', is highlighted, showing a description: 'An introduction to creating tables and...' and statistics: 'Created 8 ye...', '4' hearts, and '11.3K' views.

At the bottom of the page, there is a footer with links: 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account'. Below these links, it states '26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved'.

Table D_ORDERS created.

II) Alter table PK

```
ALTER TABLE D_ORDERS ADD CONSTRAINT PK_D_ORDERS PRIMARY KEY (ID_0);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a tree view of a database schema with tables like CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_ORDERS, MY_CUSTOMER, MY_PRODUCT, MY_PRODUCT_INFOF, ORDERS_COPY, and ORDERS_DIRECT. The main workspace is titled '[SQL Worksheet]*' and contains the SQL command: `ALTER TABLE D_ORDERS ADD CONSTRAINT PK_D_ORDERS PRIMARY KEY (ID_0);`. Below the command, the 'Script output' tab is active, showing the results of the execution: 'Table D_ORDERS created.', 'Elapsed: 00:00:00.180', 'SQL> ALTER TABLE D_ORDERS ADD CONSTRAINT PK_D_ORDERS PRIMARY KEY (ID_0)', 'Table D_ORDERS altered.', and 'Elapsed: 00:00:00.193'. The right sidebar features 'SQL Tutorials' and a 'Community' section with a search bar and a tutorial titled 'Creating...' with 4 likes and 11.3K views. The footer includes links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account', along with the version '26.4.0' and copyright information.

Table D_ORDERS altered.

b) Handle D_CUSTOMERS table

1) Create Table

```
CREATE TABLE D_CUSTOMERS AS
  SELECT CUSTOMER_ID AS ID_C, CUST_FIRST_NAME AS C_FName,
  CUST_LAST_NAME AS C_LName, GENDER AS Gender, CUST_TYPE AS Type
FROM CUSTOMER_COPY;
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a list of tables in the 'My Schema' database, including 'D_CUSTOMERS', which is currently selected. The main workspace is divided into two sections. The top section, labeled '[SQL Worksheet]*', contains the SQL script:
1 CREATE TABLE D_CUSTOMERS AS
2 ...SELECT CUSTOMER_ID AS ID_C, CUST_FIRST_NAME AS C_FName, ...
3 ...CUST_LAST_NAME AS C_LName, GENDER AS Gender, CUST_TYPE AS Type
4 FROM CUSTOMER_COPY;
The bottom section, titled 'Script output', shows the execution results:
Table D_ORDERS altered.
Elapsed: 00:00:00.193
SQL> CREATE TABLE D_CUSTOMERS AS
SELECT CUSTOMER_ID AS ID_C, CUST_FIRST_NAME AS C_FName,
CUST_LAST_NAME AS C_LName, GENDER AS Gender, CUST_TYPE AS Type
FROM CUSTOMER_COPY
Table D_CUSTOMERS created.
Elapsed: 00:00:00.161
On the right side of the interface, there is a 'SQL Tutorials' sidebar with a search bar and a list of tutorial cards. The top card is titled 'Creating...' and describes an introduction to creating tables and databases, noting it was created 8 years ago and has 4 likes and 11.3K views.

Table D_CUSTOMERS created.

II) Alter table PK

```
ALTER TABLE D_CUSTOMERS ADD CONSTRAINT PK_D_CUSTOMERS PRIMARY KEY (ID_C);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a tree view of a database schema with tables like CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_CUSTOMERS, D_ORDERS, MY_CUSTOMER, MY_PRODUCT, MY_PRODUCT_INFOF, and ORDERS_COPY. The main workspace is titled '[SQL Worksheet]*' and contains a single line of SQL code: `ALTER TABLE D_CUSTOMERS ADD CONSTRAINT PK_D_CUSTOMERS PRIMARY KEY (ID_C);`. Below the code editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing two execution results. The first result states 'Table D_CUSTOMERS created.' with an elapsed time of 00:00:00.161. The second result shows the SQL command being executed, followed by 'Table D_CUSTOMERS altered.' and an elapsed time of 00:00:00.190. On the right side of the interface, there is a sidebar with 'SQL Tutorials' and a 'Community' section featuring a search bar and a tutorial titled 'Creating...' with a description 'An introduction to creating tables and...' and statistics of 4 likes and 11.3K views.

FreeSQL >_ Worksheet SQL Tutorials 23ai Connect to the Database Help and Feedback

Navigator Files

My Schema

Tables

Search

Query result Script output DBMS output Explain Plan SQL history

Table D_CUSTOMERS created.

Elapsed: 00:00:00.161

SQL> ALTER TABLE D_CUSTOMERS ADD CONSTRAINT PK_D_CUSTOMERS PRIMARY KEY (ID_C)

Table D_CUSTOMERS altered.

Elapsed: 00:00:00.190

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account

26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

SQL Tutorials

Community My C

Search

Tutorial

Creating...

An introduction to creating tables and...

Created 8 ye...

4

11.3K

Tutorial

Table D_CUSTOMERS altered.

c) Handle D_TIME table

l) Create Table

Because T_DATE does not have ID so, we create sequence for it and apply it.

(1) Create sequence for Time ID:

```
CREATE SEQUENCE SEQ_ID_DATE START WITH 1;
```

The screenshot displays the FreeSQL web application interface. The top navigation bar includes the FreeSQL logo, a 'Worksheet' tab, 'SQL Tutorials', a user profile '23ai', and buttons for 'Connect to the Database', 'Help and Feedback', and a settings icon. On the left, a 'Navigator' sidebar shows 'My Schema' and 'Sequences' with a search bar and a list containing 'SEQ_ID_DATE'. The main workspace, titled '[SQL Worksheet]*', contains a single line of SQL code: `1. CREATE SEQUENCE SEQ_ID_DATE START WITH 1;`. Below the code editor, a tabbed interface shows 'Script output' as the active view. It displays the executed command: `SQL> CREATE SEQUENCE SEQ_ID_DATE START WITH 1`, followed by the message 'Sequence SEQ_ID_DATE created.' and the execution time 'Elapsed: 00:00:00.007'. The right sidebar features 'SQL Tutorials' links for 'Community' and 'My C...', a search bar, and a 'Tutorial' card titled 'Creating...' with a description 'An introduction to creating tables and...' and statistics '4' likes and '11.3K' views. The bottom footer contains links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account', along with the copyright notice '26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved' and the Oracle logo.

Sequence SEQ_ID_DATE created.

(2) Apply sequence to populate D_TIME from T_DATE:

```
CREATE TABLE D_TIME AS
```

```
SELECT SEQ_ID_DATE.NEXTVAL AS ID_T, ORDER_DATE AS DDate, Month_No, Month_Name, Quarter, YYear  
FROM T_DATE;
```

The screenshot shows the FreeSQL web interface. At the top, there's a navigation bar with 'FreeSQL', 'Worksheet', 'SQL Tutorials', a user profile '23ai', and buttons for 'Connect to the Database' and 'Help and Feedback'. Below this is a toolbar with various icons for editing and executing SQL. The main area displays the SQL script entered in the worksheet, which is the same as the one in the previous block. Below the script, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is selected, showing the execution results. The first result indicates that the sequence 'SEQ_ID_DATE' was created successfully, with an elapsed time of 00:00:00.009. The second result shows the execution of the 'CREATE TABLE D_TIME AS SELECT' statement, indicating that the table 'D_TIME' was created successfully, with an elapsed time of 00:00:00.041.

FreeSQL Worksheet SQL Tutorials 23ai Connect to the Database Help and Feedback

[SQL Worksheet]* [Icons] [Icons]

```
1 CREATE TABLE D_TIME AS
2 SELECT SEQ_ID_DATE.NEXTVAL AS ID_T, ORDER_DATE AS DDate, Month_No, Month_Name, Quarter, YYear FROM T_DATE;
3
```

Query result **Script output** DBMS output Explain Plan SQL history

[Icons]

Sequence SEQ_ID_DATE created.
Elapsed: 00:00:00.009

SQL> CREATE TABLE D_TIME AS
SELECT SEQ_ID_DATE.NEXTVAL AS ID_T, ORDER_DATE AS DDate, Month_No, Month_Name, Quarter, YYear FROM T_DATE

Table D_TIME created.
Elapsed: 00:00:00.041

[About Oracle](#) [Contact Us](#) [Legal Notices](#) [Terms and Conditions](#) [Your Privacy Rights](#) [Delete Your FreeSQL Account](#)

26.4.1 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Table D_TIME created.

II) Alter table PK

```
ALTER TABLE D_TIME ADD CONSTRAINT PK_D_TIME PRIMARY KEY (ID_T);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a database schema with tables like CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_CUSTOMERS, D_ORDERS, D_TIME, MY_CUSTOMER, MY_PRODUCT, and MY_PRODUCT_INFOF. The main workspace shows the SQL command: `ALTER TABLE D_TIME ADD CONSTRAINT PK_D_TIME PRIMARY KEY (ID_T);`. Below the command, the 'Script output' tab is active, showing the execution results: `70 rows inserted.`, `Elapsed: 00:00:00.328`, `SQL> ALTER TABLE D_TIME ADD CONSTRAINT PK_D_TIME PRIMARY KEY (ID_T)`, `Table D_TIME altered.`, and `Elapsed: 00:00:00.207`. The right sidebar contains 'SQL Tutorials' and a 'Community' section with a search bar and a tutorial titled 'Creating...'.

FreeSQL >_ Worksheet SQL Tutorials 23ai Connect to the Database Help and Feedback

Navigator Files

My Schema

Tables

Search

1. ALTER TABLE D_TIME ADD CONSTRAINT PK_D_TIME PRIMARY KEY (ID_T);

Query result Script output DBMS output Explain Plan SQL history

70 rows inserted.

Elapsed: 00:00:00.328

SQL> ALTER TABLE D_TIME ADD CONSTRAINT PK_D_TIME PRIMARY KEY (ID_T)

Table D_TIME altered.

Elapsed: 00:00:00.207

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account

26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

SQL Tutorials

Community My C

Search

Tutorial

Creating...

An introduction to creating tables and...

Created 8 ye...

4

11.3K

Tutorial

Table D_TIME altered.

d) Handle D_SALESMAN table

l) Create Table

```
CREATE TABLE D_SALESMAN AS  
SELECT SALES_REP_ID AS ID_S, SNAME FROM T_SALESMAN;
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' sidebar shows a tree view of the database schema, with 'D_SALESMAN' highlighted under the 'Tables' section. The main workspace is titled '[SQL Worksheet]*' and contains the SQL code: `1 CREATE TABLE D_SALESMAN AS` and `2 SELECT SALES_REP_ID AS ID_S, SNAME FROM T_SALESMAN;`. Below the code editor, the 'Script output' tab is active, showing the execution results: `SQL> CREATE TABLE D_SALESMAN AS`, `SELECT SALES_REP_ID AS ID_S, SNAME FROM T_SALESMAN`, followed by the message 'Table D_SALESMAN created.' and 'Elapsed: 00:00:00.209'. The right sidebar features 'SQL Tutorials' and a 'Community' section with a search bar and a tutorial titled 'Creating...' with 4 likes and 11.3K views. The bottom of the page includes a footer with links like 'About Oracle', 'Contact Us', and 'Legal Notices', along with the copyright notice '26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved'.

Table D_SALESMAN created.

II) Alter table PK

```
ALTER TABLE D_SALESMAN ADD CONSTRAINT PK_D_SALESMAN PRIMARY KEY (ID_S);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a tree view of a database schema with tables like CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_CUSTOMERS, D_ORDERS, D_SALESMAN, D_TIME, MY_CUSTOMER, and MY_PRODUCT. The main workspace is titled '[SQL Worksheet]*' and contains the SQL command: `ALTER TABLE D_SALESMAN ADD CONSTRAINT PK_D_SALESMAN PRIMARY KEY (ID_S);`. Below the command, the 'Script output' tab is active, showing the results of the execution: 'Table D_SALESMAN created.', 'Elapsed: 00:00:00.209', and 'SQL> ALTER TABLE D_SALESMAN ADD CONSTRAINT PK_D_SALESMAN PRIMARY KEY (ID_S)'. The output also shows 'Table D_SALESMAN altered.' and 'Elapsed: 00:00:00.189'. The right sidebar features 'SQL Tutorials' and a 'Community' section with a search bar and a tutorial titled 'Creating...'.

Table D_SALESMAN altered.

3. Create Fact Table

CREATE the fact table F_ORDERS (F_ORDERS_ID, ORDER_AMOUNT...Complete with the other columns....).

```
CREATE TABLE F_ORDERS (  
  F_ORDERS_ID NUMBER,  
  ORDER_AMOUNT NUMBER(12, 2) NOT NULL,  
  ID_T NUMBER NOT NULL,  
  ID_C NUMBER NOT NULL,  
  ID_O NUMBER NOT NULL,  
  ID_S NUMBER NOT NULL  
);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' sidebar shows a tree view of a database schema with tables like D_CUSTOMERS, D_ORDERS, D_TIME, and F_ORDERS. The 'F_ORDERS' table is selected. The main workspace is divided into two panes. The top pane, labeled '[SQL Worksheet]*', contains the SQL command: `CREATE TABLE F_ORDERS ( F_ORDERS_ID NUMBER, ORDER_AMOUNT NUMBER(12, 2) NOT NULL, ID_T NUMBER NOT NULL, ID_C NUMBER NOT NULL, ID_O NUMBER NOT NULL, ID_S NUMBER NOT NULL );`. The bottom pane, labeled 'Script output', shows the execution results: 'SQL> CREATE TABLE F_ORDERS (F_ORDERS_ID NUMBER, ORDER_AMOUNT NUMBER(12, 2) NOT NULL, ID_T NUMBER NOT NULL,... Show more...' followed by 'Table F_ORDERS created.' and 'Elapsed: 00:00:00.015'. The right sidebar features a 'SQL Tutorials' section with a search bar and a list of tutorials, including one titled 'Creating...' which is highlighted.

Table F_ORDERS created.

a) Applying PK

The PK of the fact table F_ORDERS could be either the concatenation of all FKs referencing the dimensions linked to the fact, or the UNIQUE column F_ORDERS_ID. Define a PK constraint called PK_F_ORDERS_ID on the column F_ORDERS_ID.

```
ALTER TABLE F_ORDERS ADD CONSTRAINT PK_F_ORDERS_ID PRIMARY KEY (F_ORDERS_ID);
```

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a tree view of a database schema named 'My Schema', listing tables such as CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_CUSTOMERS, D_ORDERS, D_SALESMAN, D_TIME, F_ORDERS, and MY_CUSTOMER. The main workspace is titled '[SQL Worksheet]*' and contains the following SQL code:

```
1 ALTER TABLE F_ORDERS ADD CONSTRAINT
2   PK_F_ORDERS_ID PRIMARY KEY (F_ORDERS_ID);
```

Below the code editor, the 'Script output' tab is active, showing the results of the executed queries:

- The first query, 'Table F_ORDERS created.', has an elapsed time of 00:00:00.015.
- The second query, 'SQL> ALTER TABLE F_ORDERS ADD CONSTRAINT PK_F_ORDERS_ID PRIMARY KEY (F_ORDERS_ID)', has an elapsed time of 00:00:00.028.

At the bottom of the interface, there is a footer with navigation links (About Oracle, Contact Us, Legal Notices, Terms and Conditions, Your Privacy Rights, Delete Your FreeSQL Account) and a copyright notice: '26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved'.

Table F_ORDERS altered.

b) Foreign Keys

The Fact table F_ORDERS references the four dimensions D_TIME, D_CUSTOMERS, D_ORDERS and D_SALESMAN. Define all its Foreign keys (Follow the naming convention: FK_ORDERS_TO_DATE,)

```
ALTER TABLE F_ORDERS ADD CONSTRAINT
```

```
FK_ORDERS_TO_TIME FOREIGN KEY (ID_T) REFERENCES D_TIME(ID_T);
```

```
ALTER TABLE F_ORDERS ADD CONSTRAINT
```

```
FK_ORDERS_TO_CUSTOMER FOREIGN KEY (ID_C) REFERENCES D_CUSTOMERS(ID_C);
```

```
ALTER TABLE F_ORDERS ADD CONSTRAINT
```

```
FK_ORDERS_TO_SALESMAN FOREIGN KEY (ID_S) REFERENCES D_SALESMAN(ID_S);
```

```
ALTER TABLE F_ORDERS ADD CONSTRAINT
```

```
FK_ORDERS_TO_ORDERS FOREIGN KEY (ID_O) REFERENCES D_ORDERS(ID_O);
```

The screenshot displays the FreeSQL web application interface. The top navigation bar includes the FreeSQL logo, a 'Worksheet' tab, 'SQL Tutorials', a user profile '23ai', a 'Connect to the Database' button, and links for 'Help and Feedback'. The left sidebar shows a 'Navigator' with 'My Schema' and a list of tables: CATEGORY_COPY, CURRENCY_RATES, CUSTOMER_COPY, D_CUSTOMERS, D_ORDERS, D_SALESMAN, D_TIME, F_ORDERS, and MY_CUSTOMER. The main editor area, titled '[SQL Worksheet]*', contains eight lines of SQL code for adding foreign key constraints to the F_ORDERS table. Below the editor, the 'Script output' tab is active, showing two successful execution messages: 'Table F_ORDERS altered.' with an elapsed time of 00:00:00.015. The right sidebar features 'SQL Tutorials' links, a search bar, and a featured tutorial titled 'Creating...' with a description and statistics (4 likes, 11.3K views).

[About Oracle](#) [Contact Us](#) [Legal Notices](#) [Terms and Conditions](#) [Your Privacy Rights](#) [Delete Your FreeSQL Account](#)

26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Table F_ORDERS altered.

Table F_ORDERS altered.

Table F_ORDERS altered.

Table F_ORDERS altered.

c) Sequence for Fact Table

Create an Oracle Sequence SEQ_F_ORDERS starting with 1. (We will use this sequence to populate F_ORDERS_ID primary key).

```
CREATE SEQUENCE SEQ_F_ORDERS START WITH 1;
```

The screenshot displays the FreeSQL web application interface. On the left, the 'Navigator' pane shows a tree view with 'My Schema' and 'Sequences'. Under 'Sequences', 'SEQ_F_ORDERS' is highlighted. The main editor area shows the SQL command: `1. CREATE SEQUENCE SEQ_F_ORDERS START WITH 1;`. Below the editor, the 'Script output' tab is active, displaying the following messages: `FK_ORDERS_TO_ORDERS FOREIGN KEY (ID_O) REFERENCES D_ORDERS(ID_O)`, `Table F_ORDERS altered.`, `Elapsed: 00:00:00.014`, `SQL> CREATE SEQUENCE SEQ_F_ORDERS START WITH 1`, `Sequence SEQ_F_ORDERS created.`, and `Elapsed: 00:00:00.009`. The right sidebar contains 'SQL Tutorials' and a 'Community' section with a search bar and a tutorial titled 'Creating...'.

Sequence SEQ_F_ORDERS created.

d) Load F_ORDERS

Develop SQL statement(s) to load F_ORDERS. Note that the measure ORDER_AMOUNT is in EURO and you extract it from the table ORDERS_COPY

```
INSERT INTO F_ORDERS (F_ORDERS_ID, ORDER_AMOUNT, ID_T, ID_C, ID_O, ID_S)
SELECT
    SEQ_F_ORDERS.NEXTVAL,
    OC.ORDER_TOTAL,
    DT.ID_T,
    DC.ID_C,
    DO.ID_O,
    DS.ID_S
FROM ORDERS_COPY OC
JOIN D_TIME DT ON TO_DATE(TO_CHAR(OC.ORDER_DATE, 'DD-MM-YYYY'), 'DD-MM-YYYY') = DT.DDATE
JOIN D_CUSTOMERS DC ON OC.CUSTOMER_ID = DC.ID_C
JOIN D_ORDERS DO ON OC.ORDER_ID = DO.ID_O
JOIN D_SALESMAN DS ON NVL(OC.SALES_REP_ID, 999) = DS.ID_S;
```

The screenshot displays the FreeSQL SQL Worksheet interface. The top navigation bar includes the FreeSQL logo, a 'Worksheet' tab, 'SQL Tutorials', a user profile '23ai', and buttons for 'Connect to the Database' and 'Help and Feedback'. The left sidebar features a 'Navigator' with 'My Schema' and 'Tables' sections. Under 'Tables', a list of tables is shown, with 'F_ORDERS' highlighted. The main workspace contains an SQL editor with the following code:

```
1 INSERT INTO F_ORDERS (F_ORDERS_ID, ORDER_AMOUNT, ID_T, ID_C, ID_O, ID_S)
2 SELECT
3     SEQ_F_ORDERS.NEXTVAL,
4     OC.ORDER_TOTAL,
5     DT.ID_T,
6     DC.ID_C,
7     DO.ID_O,
8     DS.ID_S
9 FROM ORDERS_COPY OC
10 JOIN D_TIME DT ON TO_DATE(TO_CHAR(OC.ORDER_DATE, 'DD-MM-YYYY'), 'DD-MM-YYYY') = DT.DDATE
11 JOIN D_CUSTOMERS DC ON OC.CUSTOMER_ID = DC.ID_C
12 JOIN D_ORDERS DO ON OC.ORDER_ID = DO.ID_O
13 JOIN D_SALESMAN DS ON NVL(OC.SALES_REP_ID, 999) = DS.ID_S;
```

Below the editor, the 'Script output' tab is active, showing the execution results:

```
SQL> INSERT INTO F_ORDERS (F_ORDERS_ID, ORDER_AMOUNT, ID_T, ID_C, ID_O, ID_S)
SELECT
    SEQ_F_ORDERS.NEXTVAL,
    OC.ORDER_TOTAL, ...
Show more...
```

The output indicates that 105 rows were inserted and the operation elapsed in 00:00:00.030 seconds. The bottom of the interface contains a footer with links to 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account', along with the version '26.4.0' and copyright information.

105 rows inserted.

e) Display Sum of the ORDER_AMOUNT

Display the SUM of the column ORDER_AMOUNT from the fact F_ORDERS

```
SELECT SUM(ORDER_AMOUNT) AS TOTAL_WH_SALES FROM F_ORDERS;
```

The screenshot shows the FreeSQL web interface. The top navigation bar includes the FreeSQL logo, a 'Worksheet' tab, 'SQL Tutorials', a user profile '23ai', a 'Connect to the Database' button, and a 'Help and Feedback' link. On the left, a 'Navigator' sidebar shows a tree view of a database schema with tables like CATEGORY_CC, CURRENCY_R, CUSTOMER_C, D_CUSTOMER, D_ORDERS, D_SALESMAN, D_TIME, F_ORDERS (highlighted), and MY_CUSTOMERS. The main area displays the SQL query: `1 SELECT SUM(ORDER_AMOUNT) AS TOTAL_WH_SALES FROM F_ORDERS;`. Below the query, tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history' are visible. The 'Query result' tab is active, showing a table with one row:

	TOTAL_WH_SALES
1	3668054.7

. The execution time is noted as 0.001 seconds. On the right, a 'SQL Tutorials' sidebar features a search bar, a 'Community' link, and a 'Tutorial' card titled 'Creating...' with a description and statistics (4 hearts, 11.3K views). The footer contains links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account'.

26.4.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

TOTAL_WH_SALES

3668054.7

1 rows selected.

f) Turnover by Customer and Year

Display the Turnover (sum of ORDER_AMOUNT) by Customer (ID_C, C_FName) and by Year.

```
SELECT C.ID_C, C.C_FName, T.YYear, SUM(F.ORDER_AMOUNT) AS Turnover
FROM F_ORDERS F
JOIN D_CUSTOMERS C ON F.ID_C = C.ID_C
JOIN D_TIME T ON F.ID_T = T.ID_T
GROUP BY C.ID_C, C.C_FName, T.YYear;
```

The screenshot shows the FreeSQL web interface. The SQL query is entered in the main editor and executed. The results are displayed in a table with the following data:

ID_C	C_FNAME	YYEAR	TURNOVER
101	Constantin	2007	107949.5
101	Constantin	2008	82445.6
102	Harrison	2007	63600.8
102	Harrison	2006	5610.6
103	Manisha	2007	6731.4
103	Manisha	2006	13860

The interface also includes a sidebar with a database navigator, a top navigation bar with 'Connect to the Database' button, and a right sidebar with 'SQL Tutorials' and 'Community' links.

ID_C	C_FNAME	YYEAR	TURNOVER
101	Constantin	2007	107949.5
101	Constantin	2008	82445.6
102	Harrison	2007	63600.8
102	Harrison	2006	5610.6
103	Manisha	2007	6731.4
103	Manisha	2006	13860
104	Harrison	2008	46257
104	Harrison	2006	94513.5
104	Harrison	2007	5835
105	Matthias	2008	37299.8
105	Matthias	2007	24076.7
106	Matthias	2008	25109.8
106	Matthias	2006	5543.1
106	Matthias	2004	5546.6

107	Matthias	2007	103141.3
107	Matthias	2006	52471.9
108	Meenakshi	2006	63518.4
108	Meenakshi	2007	149881.3
109	Christian	2008	21863
109	Christian	2007	243392.6
116	Geraldine	2007	32307
117	Guillaume	2007	153804.3
117	Guillaume	2008	126
117	Guillaume	2006	3878.4
118	Maurice	2007	100991.8
119	Maurice	2007	16447.2
120	Diane	2008	416
121	Dianne	2008	4797
144	Sivaji	2008	71173
144	Sivaji	2007	83573.8
144	Sivaji	2006	5537.8
145	Mammutti	2007	35343.2
145	Mammutti	2008	36374.7
146	Elia	2007	59213.5
146	Elia	2008	29249.1
147	Ishwarya	2006	36430.8
147	Ishwarya	2007	334847.4
148	Gustav	2007	34445.7
148	Gustav	2008	144054.8
148	Gustav	2006	7200
149	Markus	2007	343054.7
149	Markus	2008	60065
150	Goldie	2007	282694.3
151	Divine	2008	17620
152	Dieter	2007	7616.8
153	Divine	2006	48070.6
154	Frederic	2007	26632
155	Frederico	2008	23431.9
156	Goldie	2006	68501
157	Sidney	2007	7110.3
158	Frederico	2007	25270.3
159	Eddie	2007	69286.4
160	Eddie	2007	969.2
161	Ernest	2007	600
162	Ernest	2007	220
164	Charlotte	2007	1233
165	Charlotte	2007	2519
166	Dheeraj	2007	309
167	Gerard	2007	48
169	Dheeraj	2007	15760.5

170	Harry Dean	2006	66816
122	Maurice	2007	103834.4
141	Maximilian	2007	38017.8
143	Sachin	2007	27132.6
123	Elizabeth	2007	11006.2
142	Sachin	2007	25691.3
163	Ernest	2007	510
168	Hema	2008	45175

68 rows selected.

g) Count orders > value

COUNT the number of ORDERS having an amount greater than a given value (choose a value).

```
SELECT COUNT(*) FROM F_ORDERS WHERE ORDER_AMOUNT > 30000;
```

The screenshot shows the FreeSQL web interface. The main area displays the SQL query: `SELECT COUNT(*) FROM F_ORDERS WHERE ORDER_AMOUNT > 30000;`. Below the query, the 'Query result' tab is active, showing a table with one row: `COUNT(*)` with the value `35`. The execution time is 0.006 seconds. The left sidebar shows a database navigator with a tree view of tables including `CATEGORY_COPY`, `CURRENCY_RATES`, `CUSTOMER_COPY`, `D_CUSTOMERS`, `D_ORDERS`, `D_SALESMAN`, `D_TIME`, `F_ORDERS`, and `MY_CUSTOMER`. The right sidebar contains a search bar and a tutorial section titled 'Creating...' with a description: 'An introduction to creating tables and...' and a 'Tutorial' button.

COUNT(*)

35

1 rows selected.

h) Display the Percentage of amount of DIRECT Orders and ONLINE Orders.

SELECT

```
D.OMode,  
SUM(F.ORDER_AMOUNT) AS TOTAL_AMT,  
RATIO_TO_REPORT(SUM(F.ORDER_AMOUNT)) OVER () * 100 AS PERCENTAGE
```

FROM F_ORDERS F

JOIN D_ORDERS D ON F.ID_O = D.ID_O

GROUP BY D.OMode;

The screenshot shows the FreeSQL web interface. The top navigation bar includes 'FreeSQL', 'Worksheet', 'SQL Tutorials', '23ai', 'Connect to the Database', 'Help and Feedback', and a user profile icon. The left sidebar contains a 'Navigator' with 'My Schema' and 'Tables' sections, listing various database tables like CATEGORY_C, CURRENCY_R, CUSTOMER_C, D_CUSTOMERS, D_ORDERS, D_SALESMAN, D_TIME, F_ORDERS, and MY_CUSTOM. The main area displays an SQL query in a text editor, which is the same query provided in the text blocks. Below the editor, the 'Query result' tab is active, showing a table with 3 columns: OMODE, TOTAL_AMT, and PERCENTAGE. The table contains 2 rows of data. The right sidebar features 'SQL Tutorials' and 'Community' sections, with a search bar and a list of tutorials, including one titled 'Creating...'. The footer contains links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Delete Your FreeSQL Account', along with the version '26.4.0' and copyright information.

	OMODE	TOTAL_AMT	PERCENTAGE
1	DIRECT	1903629.2	51.897513960192576
2	ONLINE	1764425.5	48.102486039807424

OMODE	TOTAL_AMT	PERCENTAGE
DIRECT	1903629.2	51.89751
ONLINE	1764426.5	48.10249

2 rows selected.